AIRPORT CHAOS MONITORING SYSTEM

A PROJECT REPORT

Submitted by

Shri Jayanth N

2116241501201

in partial fulfilment of the End Semester Examination for the course

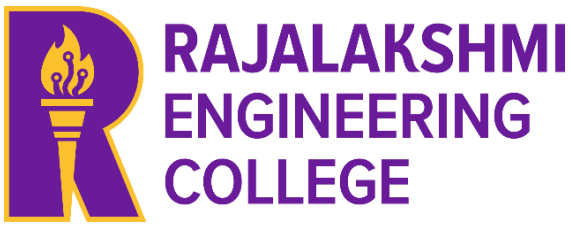
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

2026-2027

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**AIRPORT CHAOS MONITORING SYSTEM**" is the bonafide work of the student named on the cover page, who carried out the individual project work using Zoho Creator for the course **CB23F35 – Low-Code Application Development using Zoho Creator.**

SIGNATURE

Dr Ayyadurai M
HEAD OF THE DEPARTMENT
Service Type of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms Priya M
COURSE FACULTY
Assistant Professor
Service Type of Artificial Intelligence
And Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project).

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I declare that this project report titled "AIRPORT CHAOS MONITORING SYSTEM," submitted for the End Semester Examination of CB23F35 – Low-Code Application Development using Zoho Creator, is based on the work carried out by me. The report reflects the project and screenshots prepared for this application.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit.

Place: Chennai
Date: _____

SIGNATURE OF THE STUDENT
Signature of the Student

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman, Mr Meganathan M, Chairperson, Ms Thangam Meganathan and Vice-Chairman, Mr Abhay Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr Murugesan SN, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank the Head of the Department, Department of Artificial Intelligence and Machine Learning, for the guidance and support provided throughout the course.

I sincerely thank my course faculty, Ms Priya M, Assistant Professor, for her guidance, reviews and feedback during the development of this application and report.

I also acknowledge Zoho Corporation / Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

ABSTRACT

The Airport Chaos Monitoring System is a low-code application prototype developed using Zoho Creator to organise and monitor airport information, flight details, delays, and operational disruptions. The main objective of this project is to provide a centralised application that helps users record flight-related information, identify high-priority disruptions, and review operational status through reports and a dashboard.

The application consists of three main forms: Airport Details, Flight Management, and Flight Disruption. The Airport Details form stores airport information, including airport name, code, location, category, operational status, and daily flight capacity. The Flight Management form records flight numbers, airlines, origins, destinations, scheduled and actual departure times, delay duration, flight status, and disruption causes. The Flight Disruption form maintains disruption IDs, flight references, disruption types, severity levels, descriptions, resolution status, and actions taken.

The application includes a filtered report named High_Priority_Open_Disruptions to display disruptions that meet the configured priority and status conditions. The Airport Chaos Dashboard presents four summary cards showing the total number of airports, total flights, delayed flights, and total disruptions. During the demonstration, each card displayed a count of one based on the available sample records. A workflow was also configured to copy the disruption severity into the Priority field.

The prototype demonstrates how a low-code platform can be used to create structured data-entry forms, reports, dashboard components, and workflow automation with limited conventional programming. The current implementation uses demonstration data and is not connected to a verified live airport or flight data source. Therefore, the displayed information should not be interpreted as real-time operational information. The project provides a foundation for future enhancements such as live flight data integration, automated delay notifications, analytical charts, and role-based access control.

Keywords: Airport Monitoring, Zoho Creator, Low-Code Development, Flight Management, Disruption Tracking, Dashboard.

PROJECT AT A GLANCE

Attribute	Details
Project Title	Airport Chaos Monitoring System
Application Domain	Airport Operations Monitoring
Student Name & Register No.	Shri Jayanth N - 241501201
Zoho Creator Edition	Not confirmed
Application Name (Link Name)	Airport Chaos Monitoring System (Airport Chaos Dashboard)
Published / Live URL	Not available in the project details
Demo Credentials (per role)	Not included; use the assigned Zoho Creator account
Number of Forms / Subforms	3 main forms: Airport_Details, Flight_Management, and Flight_Disruption. Subforms not confirmed
Number of Reports (by type)	Standard form reports and the High_Priority_Open_Disruptions filtered report. Exact total not confirmed
Number of Pages / Front Desks	1 dashboard page: Airport Chaos Dashboard
Number of Workflows	1 configured workflow confirmed. Additional workflows or schedules not confirmed
Deluge Functions / Custom Actions	A configured workflow copies disruption severity into the Priority field. Standalone Deluge functions or custom actions not confirmed
Integrations	No external integration with live airport or flight data confirmed
Roles & Profiles Defined	Role and profile configurations not confirmed
Environments Used	Zoho Creator prototype environment. Separate development, testing, and production environments not confirmed
Date of Production Deployment	Not recorded. The application is documented as a prototype
Main Forms	Airport_Details, Flight_Management, Flight_Disruption
Main Report	High_Priority_Open_Disruptions
Dashboard Components	Total Airports, Total Flights, Delayed Flights, Total Disruptions
Workflow Purpose	Copies disruption severity into the Priority field
Demonstration Data	Sample records are used to demonstrate airport details, flight information, and disruption tracking
Observed Dashboard Values	Total Airports: 1; Total Flights: 1; Delayed Flights: 1; Total Disruptions: 1
Key Demonstration Record	DSP002 – a high-priority open disruption displayed in the filtered report

TABLE OF CONTENTS

CHAPTER	CONTENTS	PAGE
Preliminary	Abstract and Project at a Glance	iv
1	Introduction	10
1.1	Background	10
1.2	Problem Statement	10
1.3	Objectives	11
1.4	Scope of the Project	11
1.5	Justification for a Low-Code Approach	12
1.6	Organisation of the Report	12
2	Literature Survey and System Study	14
3	System Analysis and Requirements	20
4	System Design	24
5	Implementation	29
6	Testing and Debugging	38
7	Deployment, Security and Governance	43
8	Results and Discussion	46
9	Conclusion and Future Enhancements	52
A–F	Appendices	55

TABLE NO.	TITLE	PAGE
Table 2.1	Comparison of Low-Code and Traditional Development for the Proposed Application	4
Table 2.2	Summary of Literature Survey	5
Table 3.1	Stakeholders and User Roles	7
Table 3.2	Functional Requirements	7
Table 3.3	Non-Functional Requirements	9
Table 3.4	Hardware and Software Requirements	9
Table 3.5	Use Case Description	10
Table 4.1	Entities and Relationships	12
Table 4.2	Field Specification for Airport, Flight and Disruption Forms	12
Table 4.3	Role–Permission Matrix	15
Table 5.1	Reports Implemented in the Application	27
Table 5.2	Deluge Scripts Implemented	32
Table 5.3	Workflows and Blueprint Processes	35
Table 5.4	Integrations Implemented	37
Table 6.1	Functional and Workflow Test Cases	42
Table 6.2	Defect Log	44
Table 6.3	User Acceptance Feedback	44
Table 7.1	Deployment Checklist	46
Table 7.2	Performance Optimisation Results	47
Table 7.3	Security Controls Implemented	47
Table 8.1	Mapping of Objectives to Outcomes	52
Table 8.2	Existing System versus Proposed Application	53
Table A.1	Feature Coverage Checklist	56
Table B.1	Mapping of Course Outcomes to Report Chapters	60
Table E.1	Evaluation Rubric for the Individual Project	64
Table E.2	Performance Level Descriptors	65
Table F.1	Viva-Voce Questions and Responses	66
Table F.2	Marks Summary	66

LIST OF FIGURES

Figure 2.1: Existing Process Flow for Airport Operations Monitoring 14
Figure 4.1: System Architecture of Airport Chaos Monitoring System 24
Figure 4.3: Process Flow of Airport Chaos Monitoring System 27
Figure 4.4: Airport Chaos Dashboard in Zoho Creator 27
Figure 5.1: Airport Chaos Monitoring System Application Structure 29
Figure 5.2: Airport Details Form 30
Figure 5.3: Flight Management Form 30
Figure 5.4: Flight Disruption Form 30
Figure 5.6: Airport Chaos Dashboard in Zoho Creator 32
Figure 5.7: Disruption Workflow or Report Configuration 34
Figure 5.8: Application Access or User Management 36
Figure 5.9: Dashboard Metrics and Disruption Monitoring 36
Figure 5.10: Airport Chaos Monitoring System Web Interface 38
Figure 6.1: Workflow Configuration or Form Validation in Zoho Creator 41

LIST OF ABBREVIATIONS

This list defines abbreviations used in this report.

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
IT	Information Technology
JSON	JavaScript Object Notation
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
Zoho Creator	Low-code application platform

CHAPTER 1
INTRODUCTION

1.1 Background

Airport operations involve managing airport information, monitoring flight schedules, identifying delays, and tracking operational disruptions. Maintaining accurate information about these activities is important for reviewing flight status and identifying issues that require attention.

The Airport Chaos Monitoring System is a low-code application prototype developed using Zoho Creator. It provides a centralised platform to record airport details, manage flight information, and maintain records of operational disruptions. The application consists of three main forms: Airport_Details, Flight_Management, and Flight_Disruption.

The system also includes a dashboard named Airport Chaos Dashboard, which displays summary cards for total airports, total flights, delayed flights, and total disruptions. A filtered report named High_Priority_Open_Disruptions helps identify disruptions that meet the configured priority and status criteria. These features demonstrate how a low-code application can support the organisation and monitoring of airport-related information.

1.2 Problem Statement

Airport-related information includes airport details, flight schedules, delay information, and disruption records. When these records are maintained separately, reviewing the overall operational status and identifying high-priority unresolved disruptions may require additional effort.

The main problems addressed by this project are:

- Airport information, flight details, and disruption records need to be organised in a common application for easier reference.
- Flight delays and their causes need to be recorded consistently to support the review of flight status.
- Disruptions need to be classified according to their severity and tracked using resolution-status fields.
- Identifying high-priority open disruptions may require users to review multiple records unless suitable filtering is available.
- A consolidated dashboard is useful for reviewing summary counts of airports, flights, delayed flights, and disruptions.

The Airport Chaos Monitoring System addresses these information-management needs through structured forms, a filtered report, dashboard summary cards, and a configured

workflow. The current prototype uses demonstration data and is not connected to verified live airport or flight data.

1.3 Objectives

The main objectives of the Airport Chaos Monitoring System are:

- To develop a low-code application using Zoho Creator for organising airport-related information.
- To create structured forms for recording airport details, flight information, and operational disruptions.
- To record flight schedules, actual departure times, delay durations, flight status, and disruption causes.
- To maintain disruption records with severity, descriptions, resolution status, and actions taken.
- To provide a filtered report named High_Priority_Open_Disruptions for reviewing disruptions that meet the configured criteria.
- To develop an Airport Chaos Dashboard containing summary cards for total airports, total flights, delayed flights, and total disruptions.
- To configure a workflow that copies disruption severity into the Priority field.
- To demonstrate the use of low-code forms, reports, dashboards, and workflow automation in an airport operations monitoring prototype.

1.4 Scope of the Project

The scope of this project covers the development of a prototype for organising airport information, managing flight records, and monitoring operational disruptions using Zoho Creator.

The Airport_Details form stores information such as airport name, airport code, city, country, category, operational status, and daily flight capacity. The Flight_Management form records flight numbers, airline names, origins, destinations, flight dates, scheduled and actual departure times, delay durations, flight status, and disruption causes.

The Flight_Disruption form records disruption IDs, related flight numbers, disruption types, severity levels, dates, descriptions, resolution status, and actions taken. The application also includes the High_Priority_Open_Disruptions report and the Airport Chaos Dashboard.

The prototype demonstrates how structured data entry, filtered reporting, summary cards, and workflow configuration can support operational information monitoring. Its current scope is limited to demonstration data. Live flight tracking, external API integration, automated notifications, and production deployment have not been established as implemented features.

These can be considered for future development.

1.5 Justification for a Low-Code Approach

Zoho Creator was selected for this project because its low-code development environment supports the creation of data-entry forms, reports, application pages, and workflows through visual configuration. These capabilities are suitable for developing a prototype that organises airport details, flight records, and disruption information.

The form builder supports structured data capture, while reports allow records to be displayed and filtered according to defined criteria. Dashboard components provide a consolidated view of selected information, and workflow configuration can automate specific actions within the application.

Using a low-code platform reduces the need to develop every application component from scratch using conventional programming. It also allows application components to be configured and modified within the platform. However, correct data modelling, validation, testing, and access-control configuration remain important.

For this project, Zoho Creator provides a practical platform for demonstrating airport information management through a prototype. Real-time flight data integration, advanced analytics, and additional operational automation would require further implementation and verification.

1.6 Organisation of the Report

Chapter 1 introduces the Airport Chaos Monitoring System, including its background, problem statement, objectives, scope, and justification for using a low-code development platform.

Chapter 2 presents the literature survey and system study, covering the existing approach to airport information monitoring and the suitability of low-code development.

Chapter 3 describes the system requirements, including stakeholders, functional requirements, non-functional requirements, hardware and software requirements, and the use case model.

Chapter 4 explains the system design, including the architecture, data model, form design, workflow design, user interface, and access-control considerations.

Chapter 5 describes the implementation of the application in Zoho Creator, including forms, reports, dashboard components, and workflow configuration.

Chapter 6 presents the testing strategy, test cases, debugging considerations, defect documentation, and user acceptance testing requirements.

Chapter 7 discusses deployment, security, performance, and governance considerations for

the application.

Chapter 8 presents the demonstrated application outputs, evaluates the achievement of the project objectives, and discusses the limitations of the current prototype.

Chapter 9 concludes the project and outlines future enhancements, including live flight data integration, automated notifications, analytical charts, and additional access-control features.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

Airport operations involve maintaining airport information, recording flight schedules, monitoring delays, and tracking operational disruptions. For this project, the existing system is considered a general information-management process in which airport and flight details may be recorded in separate records or spreadsheets. No specific legacy airport application or measured operational baseline was supplied. Therefore, this description is used as an analytical baseline rather than an audited description of a particular airport.

In a typical process, flight information is collected, scheduled and actual departure times are reviewed, delays are recorded, and operational disruptions are documented. Staff may then review the records to identify high-priority issues and prepare summaries for monitoring purposes.

The baseline process can be represented as follows:

Airport details are recorded → Flight information is maintained → Flight delays are identified → Disruptions are recorded → Priority and resolution status are reviewed → Reports are prepared for monitoring.

When information is maintained separately, reviewing the overall status may require users to consult multiple records. The proposed Airport Chaos Monitoring System demonstrates a centralised approach to organising this information using Zoho Creator.

Figure 2.1: Existing Process Flow for Airport Operations Monitoring

Airport details are recorded → Flight information is maintained → Flight delays are identified → Disruptions are recorded → Priority and resolution status are reviewed → Reports are prepared for monitoring.

2.2 Limitations of the Existing System

The following limitations may occur when airport and flight information is maintained through separate records or manual processes:

- Airport, flight, and disruption information may be stored in different locations, making it difficult to obtain a consolidated view.
- Flight delays and their causes may require manual review if the information is not organised consistently.
- Identifying high-priority unresolved disruptions may require staff to filter or examine multiple records.
- Preparing operational summaries may take additional effort when information must be combined manually.
- Without integration with a verified live flight data source, the system cannot automatically provide real-time flight status or delay updates.

The proposed prototype addresses some of these information-management challenges through structured forms, a filtered report, dashboard summary cards, and a configured workflow.

2.3 Low-Code versus Traditional Development

The Airport Chaos Monitoring System was developed using Zoho Creator, a low-code application development platform. It provides visual tools for creating forms, reports, application pages, and workflows. These features are suitable for developing a prototype that organises airport details, flight information, and disruption records.

In traditional development, the database, user interface, authentication, hosting, and automation components may require separate implementation and integration. In a low-code environment, many application components can be configured visually, while scripting can be used where necessary.

The comparison below is qualitative. No measured development-time or cost comparison was recorded for this project.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development Approach	Requires programming to develop application components.	Forms, reports, pages, and workflows can be configured visually.
Development Time	Depends on application complexity and development effort.	Visual configuration can reduce the amount of custom code required.
Skills Required	Programming, database design, application development, and deployment.	Platform configuration, data modelling, workflow design, and scripting where required.
Hosting and Infrastructure	Hosting and infrastructure must be selected and configured.	Hosting is provided through the platform, subject to the selected plan configuration.
Mobile Access	Requires responsive design or mobile application development.	Depends on platform capabilities and application configuration.
Security and Access Control	Authentication and authorisation must be designed and configured.	Platform access-control features can be configured to meet application requirements.
Maintenance and Changes	Changes may require code modifications, testing, and deployment.	Many form and report changes can be made through visual configuration.
Customisation	Offers broad control over application architecture and implementation.	Depends on platform capabilities and supported scripting.

Cost	Depends on development, hosting, and maintenance costs.	Depends on subscription, usage, and configuration effort.

The comparison shows that low-code development can simplify the creation of standard application components. However, it does not eliminate the need for proper data modelling, validation, testing, security configuration, and maintenance. For this project, Zoho Creator provides a practical environment for demonstrating forms, reports, dashboard components, and workflow configuration without developing the entire application from scratch.

2.4 Literature Survey

The literature survey considers low-code application development and the platform features relevant to the Airport Chaos Monitoring System. Low-code development platforms provide visual tools that help developers create business applications with less conventional programming. However, the suitability of a low-code approach depends on application complexity, customisation requirements, governance, and platform limitations.

Official Zoho Creator documentation provides guidance on creating forms, reports, workflows, and application pages. These capabilities are relevant to the implementation of the Airport_Details, Flight_Management, and Flight_Disruption forms, as well as the Airport Chaos Dashboard and the High_Priority_Open_Disruptions report.

Table 2.2: Summary of Literature Survey

S. No.	Author / Source	Title / Topic	Key Findings	Relevance to This Project
1	Zoho Corporation	Zoho Creator Documentation	Describes the platform's application development capabilities, including forms, reports, pages, and workflows.	Provides the platform background for developing the prototype.
2	Zoho Corporation	Forms and Data Management	Explains how structured forms can be used to collect and manage application records.	Relevant to the Airport_Details, Flight_Management, and Flight_Disruption forms.
3	Zoho Corporation	Reports in Zoho Creator	Describes methods for displaying,	Relevant to the High_Priority_Open_Disruptions report.

			organising, and filtering application records.	
4	Zoho Corporation	Workflows in Zoho Creator	Explains how configured workflows can automate actions based on defined events and conditions.	Relevant to the workflow that copies disruption severity into the Priority field.
5	Zoho Corporation	Pages and Dashboard Components	Describes the presentation of application information through pages and visual components.	Relevant to the Airport Chaos Dashboard and its four summary cards.

2.5 Emerging Trends in Low-Code Development

Low-code development continues to support the creation of business applications through visual builders, reusable components, workflow automation, and dashboard-based information presentation. These capabilities help developers organise data and implement common business processes without writing every component from scratch.

Another area of development is the use of application programming interfaces (APIs) to connect applications with external data sources. For airport monitoring, integration with a reliable flight data provider could support more current flight information and delay updates. Automated notifications, trend analysis, and advanced analytics could also improve future versions of the application.

The current Airport Chaos Monitoring System demonstrates visual form development, filtered reporting, dashboard summary cards, and workflow configuration. It uses demonstration data and does not have a verified live flight data integration. AI-based functionality, advanced analytics, and automated live-data updates are therefore considered future enhancements rather than implemented features.

2.6 Proposed System

The proposed Airport Chaos Monitoring System is a low-code application prototype developed using Zoho Creator. Its purpose is to centralise airport information, maintain flight records, and track operational disruptions through structured forms, filtered reports, a dashboard, and workflow configuration.

The application contains three main forms:

1. **Airport_Details:** Stores airport name, airport code, city, country, category, operational status, and daily flight capacity.
2. **Flight_Management:** Stores flight number, airline, origin, destination, flight date, scheduled and actual departure times, delay duration, flight status, and disruption cause.
3. **Flight_Disruption:** Stores disruption ID, related flight number, disruption type, severity, date, description, resolution status, and action taken.

The application includes a filtered report named **High_Priority_Open_Disruptions**, which displays records that meet the configured priority and status criteria. The **Airport Chaos Dashboard** contains four summary cards: Total Airports, Total Flights, Delayed Flights, and Total Disruptions.

A configured workflow copies disruption severity into the Priority field. During the demonstration, the dashboard displayed one airport, one flight, one delayed flight, and one disruption. The DSP002 record was displayed in the high-priority open disruptions report.

The proposed system demonstrates how low-code development can support structured information management and operational monitoring. However, it remains a prototype using demonstration data and does not provide verified real-time airport or flight information. Integration with live flight data, automated notifications, analytical charts, and additional access-control features can be considered in future development.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Stakeholders and User Roles

The Airport Chaos Monitoring System is designed to help airport operations personnel maintain airport information, monitor flight delays, record disruptions, and identify high-priority unresolved issues. The following table describes the main stakeholder roles considered for the proposed system. These are conceptual roles; separate Zoho Creator permission profiles have not been verified.

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Responsible for overall application administration.	Maintain airport, flight, and disruption records; review dashboard information; manage application configuration.
Airport Operations Staff	Personnel responsible for monitoring airport operations.	Review flight details, update flight statuses, and monitor flight delays.
Operations Supervisor	Personnel responsible for reviewing significant operational issues.	Review high-priority disruptions, monitor resolution status, and coordinate follow-up actions.
Monitoring Staff	Personnel responsible for maintaining operational records.	Record flight disruptions, update descriptions and statuses, and review disruption reports.

3.2 Functional Requirements

Functional requirements describe the main operations that the Airport Chaos Monitoring System is intended to support. The requirements below are based on the forms, reports, and dashboard included in the Zoho Creator prototype.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	Store airport information, including airport name, code, city, country, category, operational status, and daily flight capacity.	Administrator	High
FR-	Store flight details, including flight number,	Administrator, Airport	High

02	airline, origin, destination, flight date, scheduled and actual departure times, delay duration, status, and disruption cause.	Operations Staff	
FR-03	Record flight disruptions with a disruption ID, related flight number, disruption type, severity, date, description, and resolution status.	Airport Operations Staff, Monitoring Staff	High
FR-04	Record actions taken to address flight disruptions and update their resolution status.	Airport Operations Staff, Operations Supervisor	High
FR-05	Display summary metrics for total airports, total flights, delayed flights, and total disruptions on the dashboard.	Administrator, Airport Operations Staff, Operations Supervisor	High
FR-06	Provide reports to review flight and disruption records.	Airport Operations Staff, Monitoring Staff, Operations Supervisor	Medium
FR-07	Display high-priority unresolved disruptions through a filtered report for easier monitoring.	Operations Supervisor, Airport Operations Staff	High

The above requirements describe the intended application behavior. The prototype contains the corresponding airport, flight, and disruption forms, dashboard metrics, and a high-priority unresolved disruptions report. Full operational testing and access permissions should be verified before production use.

3.3 Non-Functional Requirements

Non-functional requirements describe the expected quality, usability, security, and maintainability of the application.

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Dashboard metrics and reports should load within an acceptable time when working with representative airport and flight records. No performance threshold has been measured.
NFR-02	Security	Access to airport, flight, and disruption information should be restricted through appropriate user permissions before

		operational deployment.
NFR-03	Usability	Forms, reports, and dashboard elements should use clear labels and a consistent layout to support easy navigation.
NFR-04	Availability	Application availability depends on the Zoho Creator service, account status, and network connectivity. No project-specific availability target has been established.
NFR-05	Maintainability	Forms, fields, reports, and workflows should use consistent naming conventions and be documented to support future modifications.
NFR-06	Data Accuracy	Airport, flight, delay, and disruption information should be entered and maintained accurately. The prototype uses demonstration data and is not connected to verified live flight information.

3.4 Hardware and Software Requirements

The Airport Chaos Monitoring System is developed as a low-code application using Zoho Creator. The application is accessed through a web browser, while Deluge is the scripting language available within the platform. The prototype uses demonstration records and does not include a verified live flight API integration.

Table 3.4: Hardware and Software Requirements

Category	**Specification**
Development Platform	Zoho Creator
Development Approach	Low-code application development
Scripting Language	Deluge, where required for application logic
Web Browser	A browser supported by Zoho Creator; exact version not recorded
Operating System	Compatible operating system capable of running a supported web browser
Client Hardware	Internet-connected computer capable of running the application; minimum hardware requirements not measured
Database	Zoho Creator form-based data storage
External API	No live flight API integration implemented in the prototype

Other External Services	No external service connection verified
Mobile Access	Not independently tested
Internet Connection	Required to access the cloud-based application

3.5 Use Case Model

The use case model describes the main interactions between users and the Airport Chaos Monitoring System. The conceptual actors are the Administrator, Airport Operations Staff, Operations Supervisor, and Monitoring Staff. Their actual access permissions must be configured and verified separately in Zoho Creator.

Table 3.5: Use Case Description

Field	**Description**
Use Case ID	UC-01
Use Case Name	Record and Monitor a Flight Disruption
Actor(s)	Airport Operations Staff, Monitoring Staff
Pre-condition	The user can access the Zoho Creator application, and the required permissions have been configured.
Main Flow	1. Open the Flight Disruption form. 2. Enter the disruption ID and related flight number. 3. Select the disruption type and severity. 4. Enter the date and description. 5. Set the resolution status and record any action taken. 6. Submit the record and review it in the disruption report.
Alternate Flow	If information is missing or incorrect, the user should correct the relevant fields before completing the record. Form validation behavior must be verified through testing.
Post-condition	The disruption record is available for review in the application, subject to successful submission and the configured report criteria.
Related Functional Requirements	FR-03, FR-04, FR-06, FR-07

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The Airport Chaos Monitoring System is developed using Zoho Creator, a low-code application development platform. The application provides forms for maintaining airport details, managing flight information, and recording flight disruptions. Reports and a dashboard help users review flight delays and identify high-priority unresolved disruptions.
The system can be represented using three logical layers. The presentation layer provides the Zoho Creator interface through which users access forms, reports, and dashboard pages. The application layer contains the forms, report criteria, dashboard metrics, and configured workflow logic. The data layer stores the airport, flight, and disruption records in Zoho Creator. The prototype uses demonstration data and is not connected to a verified live flight information service.

Figure 4.1: System Architecture of Airport Chaos Monitoring System



1. **Presentation Layer:** Zoho Creator web interface, forms, reports, and dashboard.
2. **Application Layer:** Airport Details, Flight Management, Flight Disruption, report filters, dashboard metrics, and configured workflow.
3. **Data Layer:** Airport records, flight records, and flight disruption records stored in Zoho Creator.

The arrows should indicate that users interact with the presentation layer, which uses the application components to retrieve and update stored records.

4.2 Data Model

The data model of the Airport Chaos Monitoring System consists of three main entities: Airport Details, Flight Management, and Flight Disruption. Each entity stores information required for airport operations monitoring.

The Airport Details form stores airport identification and operational information. The Flight Management form stores flight schedules, departure details, delay duration, flight status, and disruption cause. The Flight Disruption form records disruption type, severity, description, resolution status, and action taken.

The flight number recorded in the disruption form can be used to identify the related flight. The precise lookup configuration and enforcement of relationships should be verified in Zoho Creator before production use.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Airport Details	Airport Name, Airport Code, City, Country, Category, Operational Status, Daily Flight Capacity	Flight Management	One airport may be associated with multiple flights conceptually	Airport and flight records; lookup configuration to be verified
Flight Management	Flight Number, Airline, Origin, Destination, Flight Date, Scheduled Departure, Actual Departure, Delay Duration, Flight Status, Disruption Cause	Airport Details, Flight Disruption	A flight may be associated with one or more disruption records conceptually	Flight form fields; relationship configuration to be verified
Flight Disruption	Disruption ID, Flight Number, Disruption Type, Severity, Date, Description, Resolution Status, Action Taken	Flight Management	Multiple disruptions may refer to the same flight	Flight number field and disruption records; lookup configuration to be verified

4.3 Form Design

The application uses three main forms to collect and organize operational information. The Airport Details form captures airport identification and capacity information. The Flight Management form stores flight schedules, actual departure details, delay duration, and flight status. The Flight Disruption form captures information about operational disruptions and their resolution.

The forms provide structured fields so that records can be reviewed through reports and summarized on the dashboard.

Table 4.2: Field Specification for Airport, Flight and Disruption Forms

Field Label	Link Name / Field	Field Type	Mandatory	Validation / Default	Remarks
Airport Name	Airport name field	Text	Not verified	Not verified	Identifies the airport
Airport Code	Airport code field	Text	Not verified	Not verified	Stores the airport code
City / Country	City and country fields	Text	Not verified	Not verified	Records the airport location
Operational Status	Operational status field	Choice / picklist	Not verified	Configured options to be checked	Indicates airport operating status
Daily Flight Capacity	Daily flight capacity field	Numeric	Not verified	Not verified	Stores daily flight capacity
Flight Number	Flight number field	Text	Not verified	Not verified	Identifies the flight
Airline	Airline field	Text or choice	Not verified	Not verified	Identifies the airline
Scheduled Departure	Scheduled departure field	Date-time or time	Not verified	Not verified	Stores scheduled departure information
Actual Departure	Actual departure field	Date-time or time	Not verified	Not verified	Stores actual departure information
Delay Duration	Delay duration field	Numeric	Not verified	Example record: 45 minutes	Stores delay duration in minutes
Flight Status	Flight status field	Choice / picklist	Not verified	Includes a Delayed status in the prototype	Used for delayed-flight reporting
Disruption ID	Disruption ID field	Text or identifier	Not verified	Example: DSP002	Identifies a disruption record
Severity	Severity field	Choice / picklist	Not verified	Includes a high-severity example	Indicates disruption severity
Resolution Status	Resolution status field	Choice / picklist	Not verified	Example records include Monitoring and an open disruption	Tracks the disruption status
Action Taken	Action taken field	Multiline text	Not verified	Not verified	Records the response to the disruption

4.4 Process and Workflow Design

The main process begins when airport information is entered into the Airport Details form. Flight information is then recorded in the Flight Management form, including scheduled and actual departure details and any delay information. When an operational issue occurs, a corresponding record is entered into the Flight Disruption form.

Disruption records can be reviewed through reports. High-priority unresolved disruptions are displayed in the filtered report, allowing operations personnel to identify issues requiring attention. The dashboard summarizes airport, flight, delayed-flight, and disruption counts.

The prototype also includes configured workflow logic intended to copy disruption severity into a Priority field. The exact trigger conditions and behavior should be verified before describing the workflow as fully tested.

Figure 4.3: Process Flow of Airport Chaos Monitoring System

Start → Enter Airport Details → Record Flight Information → Update Flight Status and Delay Details → Record Flight Disruption, if applicable → Review Disruption Report → Identify High-Priority Unresolved Issues → View Dashboard → End.

4.5 User Interface and Navigation Design

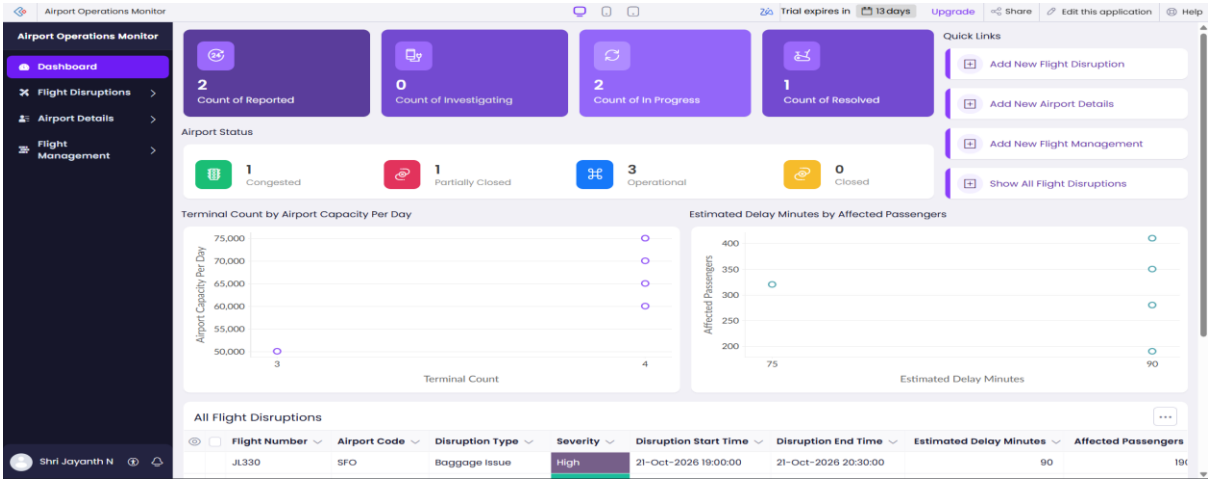
The user interface is developed using Zoho Creator's web interface. The application provides separate forms for airport details, flight management, and flight disruptions. The reports allow users to view stored records, while the Airport Chaos Dashboard presents summary metrics for monitoring the application data.

The dashboard contains four statistic cards:

- Total Airports
- Total Flights
- Delayed Flights
- Total Disruptions

A high-priority unresolved disruptions report is also included on the dashboard. During the recorded demonstration, each statistic displayed a count of 1. These values reflect the demonstration data at that time and are not live airport statistics.

Figure 4.4: Airport Chaos Dashboard in Zoho Creator



4.6 Access Control Design

The Airport Chaos Monitoring System is intended for use by personnel involved in airport operations monitoring. Different responsibilities may require different levels of access to airport details, flight information, and disruption records.
The following matrix represents a proposed access-control design. It does not confirm that separate user permission profiles have already been configured in Zoho Creator. Actual form, report, record, and page permissions must be checked before operational deployment.

Table 4.3: Role–Permission Matrix

Component	Administrator	Airport Operations Staff	Operations Supervisor	Monitoring Staff
Airport Details	Proposed: Add, Edit, View, Delete	Proposed: View	Proposed: View	Proposed: View
Flight Management	Proposed: Add, Edit, View, Delete	Proposed: Add, Edit, View	Proposed: View	Proposed: View
Flight Disruption	Proposed: Add, Edit, View, Delete	Proposed: Add, Edit, View	Proposed: View, Review	Proposed: Add, Edit, View
Standard Reports	Proposed: View and Manage	Proposed: View	Proposed: View	Proposed: View
High-Priority Unresolved Disruptions Report	Proposed: View and Manage	Proposed: View	Proposed: View and Review	Proposed: View
Airport Chaos Dashboard	Proposed: View and Manage	Proposed: View	Proposed: View	Proposed: View

CHAPTER 5
IMPLEMENTATION

This chapter describes the implementation of the Airport Chaos Monitoring System using Zoho Creator. The application prototype includes airport information management, flight information management, flight disruption records, reports, dashboard metrics, and configured workflow logic. The implementation details presented in this chapter are based on the available application information and observed demonstration records. Features that have not been verified are identified accordingly.

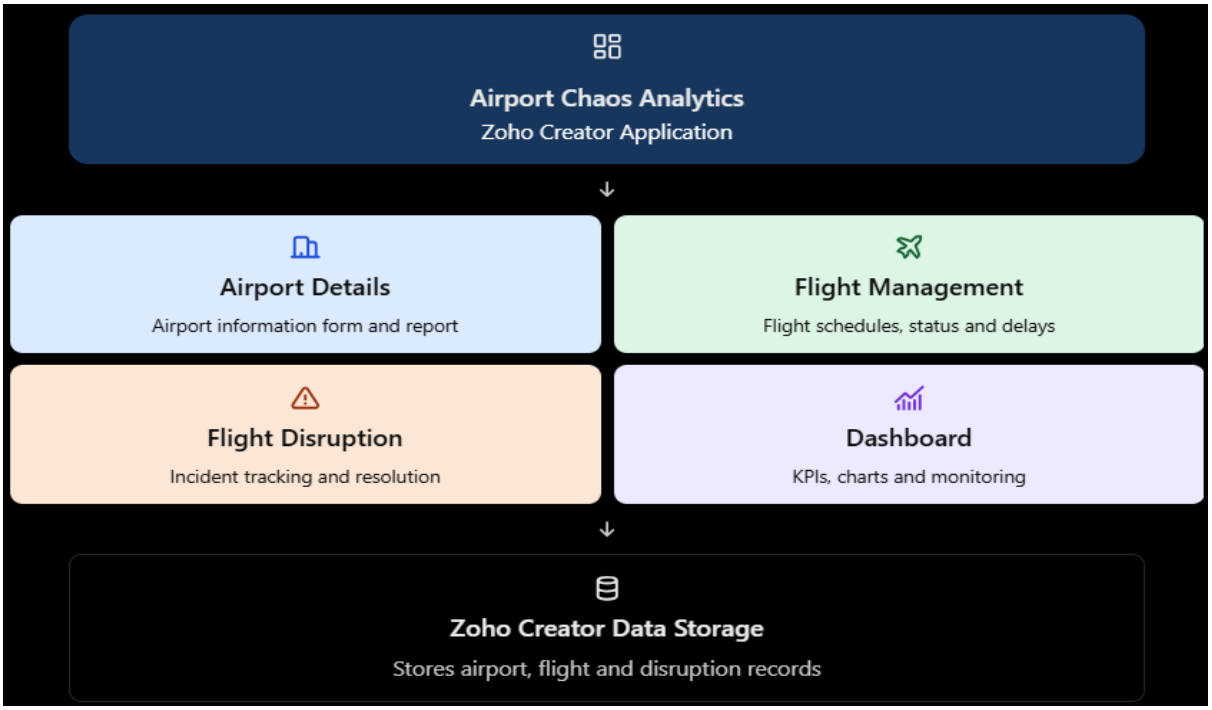
5.1 Application Setup

The Airport Chaos Monitoring System was developed using Zoho Creator, a low-code application development platform. The application organizes airport information, flight details, and disruption records through separate forms. Reports and a dashboard provide a consolidated view of the stored information.
The main components of the application are:

- Airport Details form
- Flight Management form
- Flight Disruption form
- Standard application reports
- High-Priority Open Disruptions report
- Airport Chaos Dashboard
- Configured workflow logic for disruption priority

The prototype uses demonstration data and is not connected to a verified live airport or flight information service.

Figure 5.1: Airport Chaos Monitoring System Application Structure



5.2 Forms and Data Capture

Forms are the primary data-entry components of the application. Three main forms are used to organize information related to airports, flights, and operational disruptions.

The Airport Details form stores airport name, airport code, city, country, category, operational status, and daily flight capacity.

The Flight Management form stores flight number, airline, origin, destination, flight date, scheduled departure, actual departure, delay duration, flight status, and disruption cause. The demonstration data includes flight AI573 with a recorded delay of 45 minutes.

The Flight Disruption form stores the disruption ID, flight number, disruption type, severity, date, description, resolution status, and action taken. Example records include DSP001 and DSP002, which demonstrate disruption tracking and report filtering.

Figure 5.2: Airport Details Form

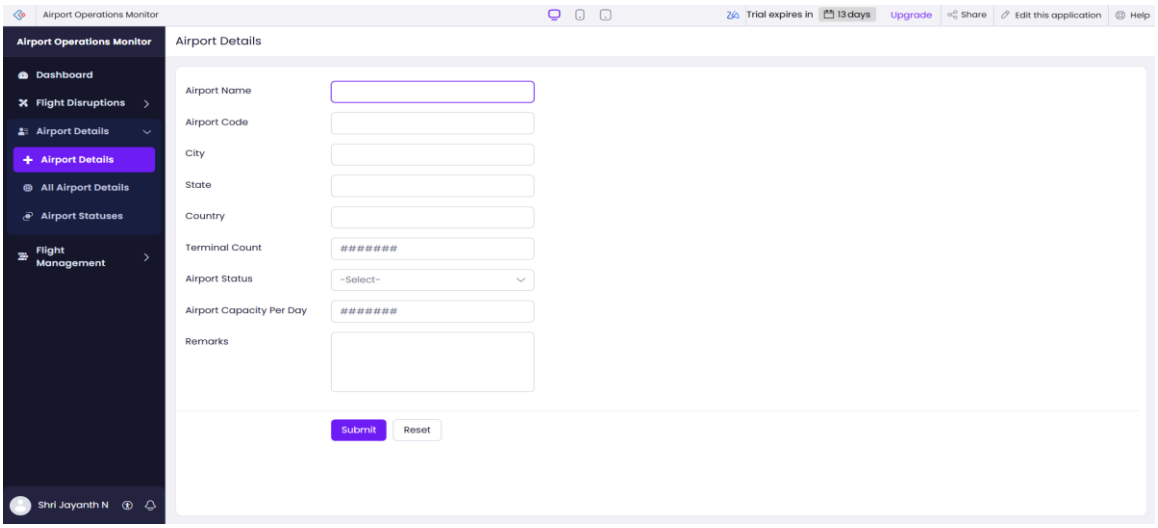


Figure 5.3: Flight Management Form

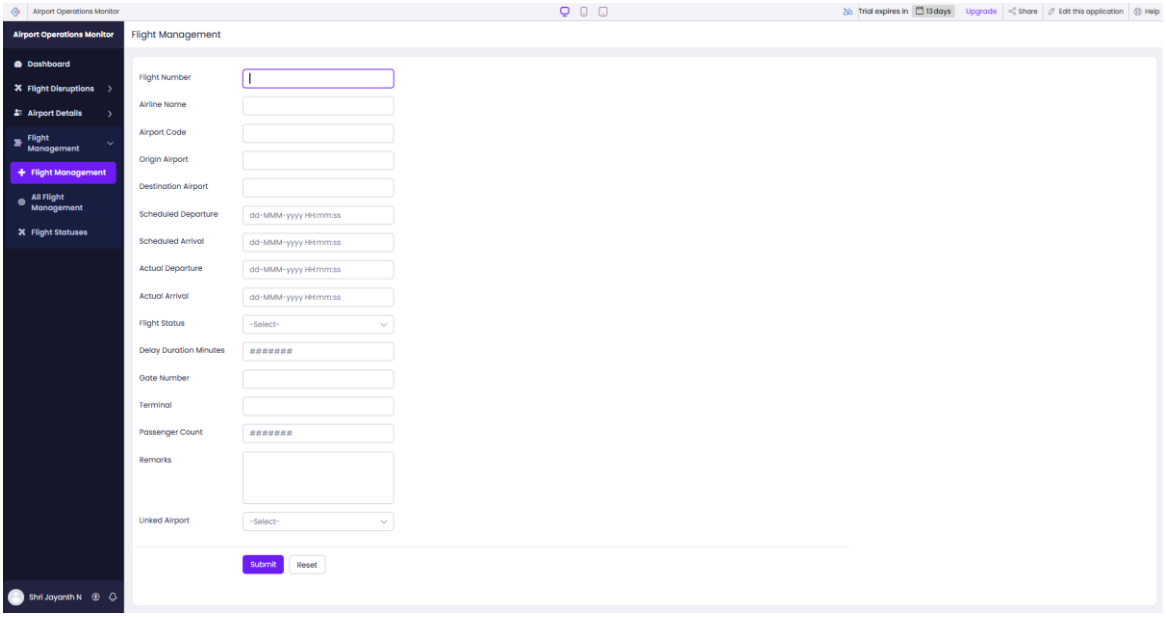
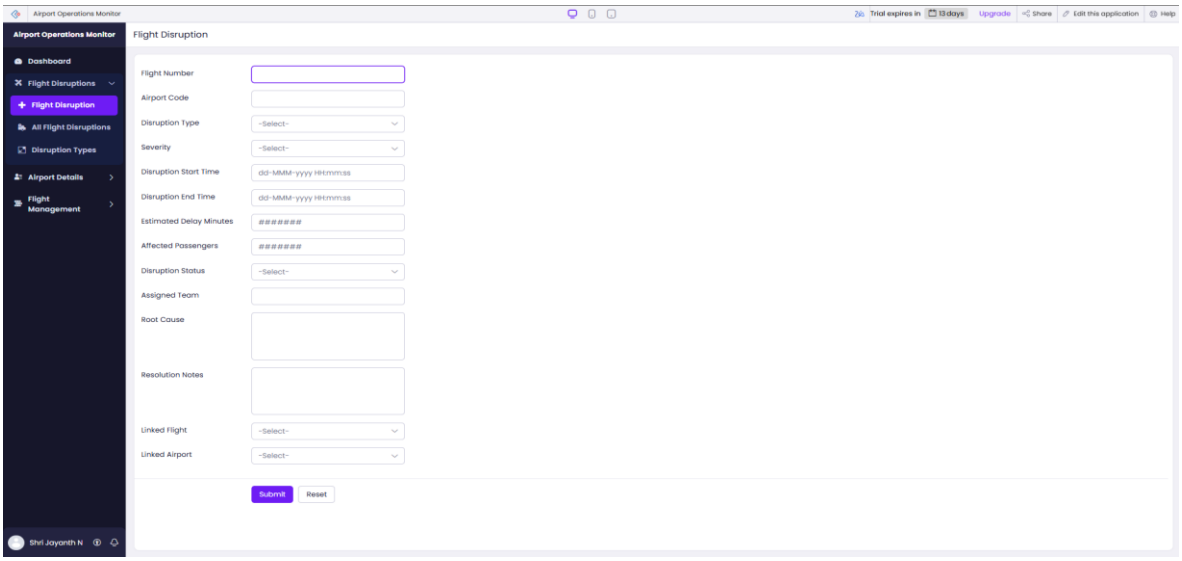


Figure 5.4: Flight Disruption Form



These forms provide structured storage for operational information and support the reports and dashboard described in the following sections. Field validation and mandatory-field behavior should be documented only where they have been verified in the application.

5.3 Reports

Reports are used to view and review the records stored in Zoho Creator. The application includes reports for the airport, flight, and disruption information maintained through the forms.
The High-Priority Open Disruptions report is configured to display disruption records that meet its filtering criteria. During the demonstration, record DSP002 appeared in this report. This demonstrates the report's intended use for identifying high-priority unresolved issues in the sample data.

Table 5.1: Reports Implemented in the Application

Report Name	Type	Source Form	Filter / Grouping / Sorting	Purpose
Airport Details Report	Record list	Airport Details	Based on available airport records	Review airport information
Flight Management Report	Record list	Flight Management	Based on available flight records	Review flight schedules and statuses
Flight Disruption Report	Record list	Flight Disruption	Based on available disruption records	Review disruption details and resolution status
High-Priority Open Disruptions	Filtered report	Flight Disruption	Filters high-priority unresolved disruptions	Identify issues requiring attention

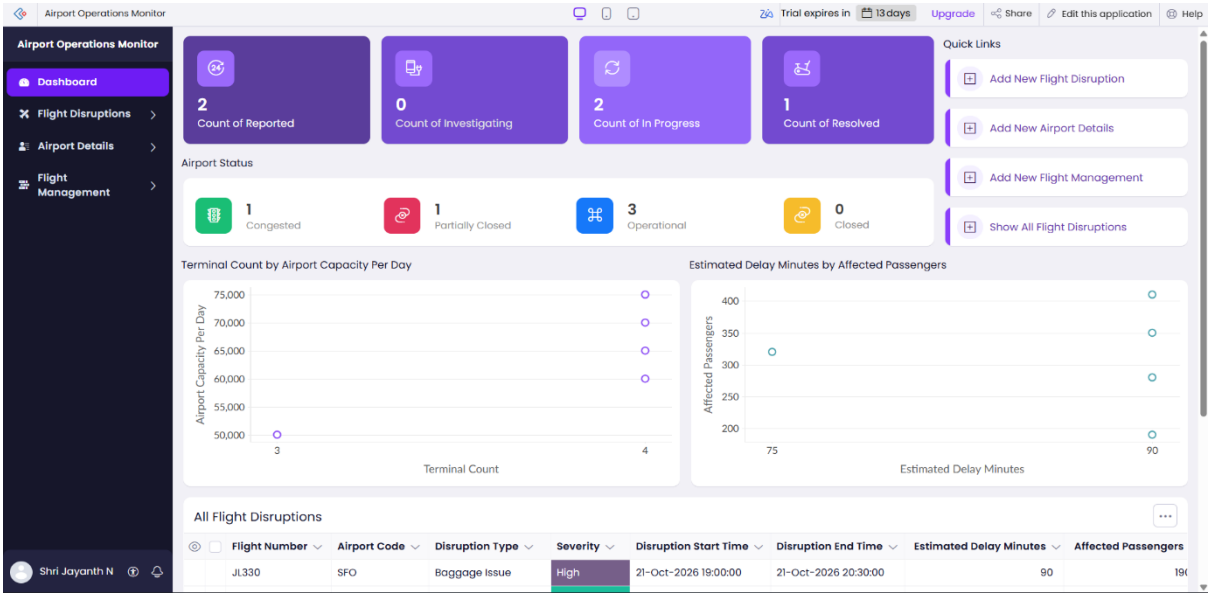
5.4 Pages and Dashboards

The Airport Chaos Dashboard provides a consolidated view of important application metrics. It contains four statistic cards that summarize the airport, flight, delay, and disruption records available in the application.
The dashboard includes the following metrics:

1. **Total Airports:** Counts the records in the Airport Details form.
2. **Total Flights:** Counts the records in the Flight Management form.
3. **Delayed Flights:** Counts flight records whose status is Delayed.
4. **Total Disruptions:** Counts the records in the Flight Disruption form.

The dashboard also includes the High-Priority Open Disruptions report, allowing users to review important unresolved issues alongside the summary metrics.
During the recorded demonstration, all four statistic cards displayed a value of 1. These values represent the sample data available at that time and should not be interpreted as actual airport operating statistics.

Figure 5.6: Airport Chaos Dashboard in Zoho Creator



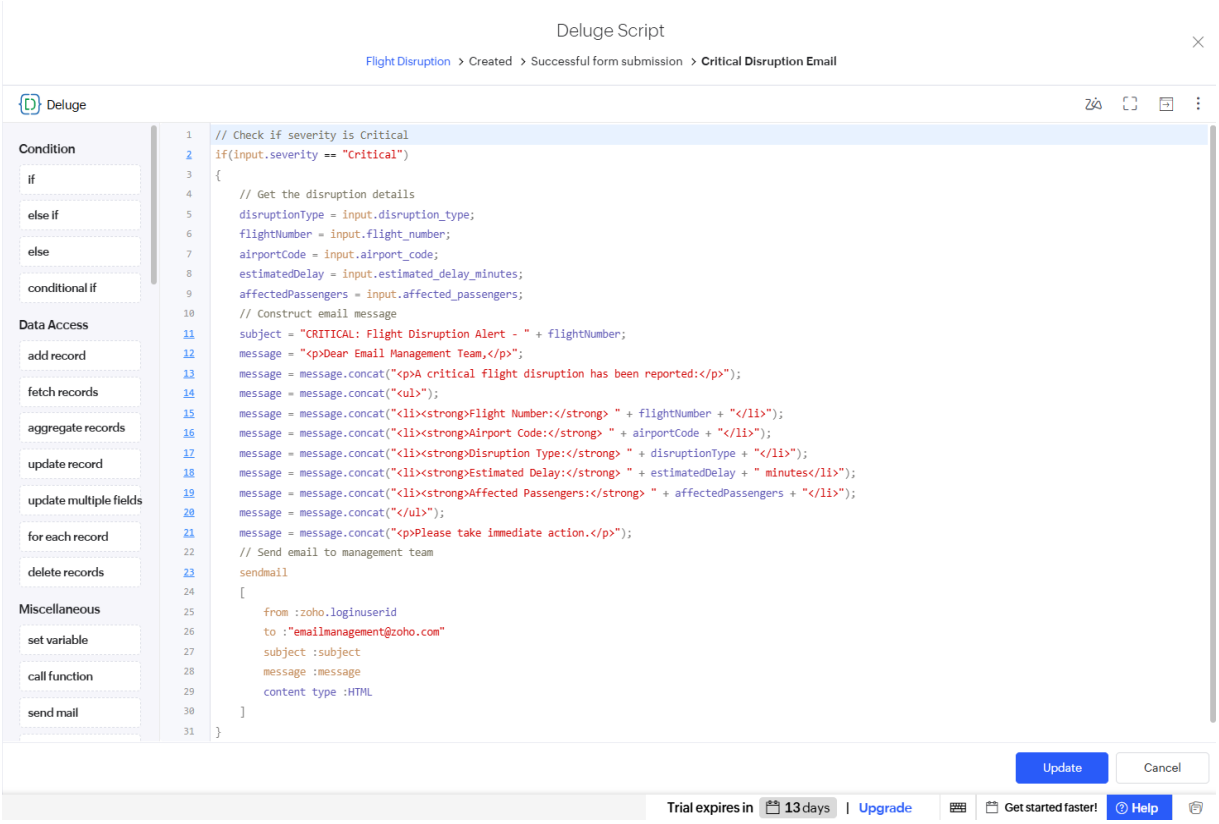
5.5 Deluge Scripting

Zoho Creator supports Deluge, a scripting language used to implement application logic and automate tasks. The Airport Chaos Monitoring System includes configured workflow logic intended to copy the severity of a disruption into a Priority field.
This logic is intended to support consistent identification of high-priority disruptions. The exact trigger, field mapping, execution behavior, and error handling should be verified in the application before describing the workflow as fully tested.
No additional standalone Deluge functions or scripts are claimed unless their source code is available and verified.

Table 5.2: Deluge Scripts and Logic

Script ID	Location / Trigger	Purpose	Implementation Status
DS-01	Disruption-related workflow; exact trigger to be verified	Copy disruption severity into the Priority field	Configured logic reported; execution details require verification
DS-02	No separate script verified	No additional standalone script identified from the available project information	Not verified
DS-03	No separate function verified	No standalone function claimed	Not verified

Code 5.1: Disruption Severity and Priority Logic



The configured logic is intended to support priority-based monitoring. Its results should be checked by entering or updating a disruption record and confirming that the Priority field receives the expected value.

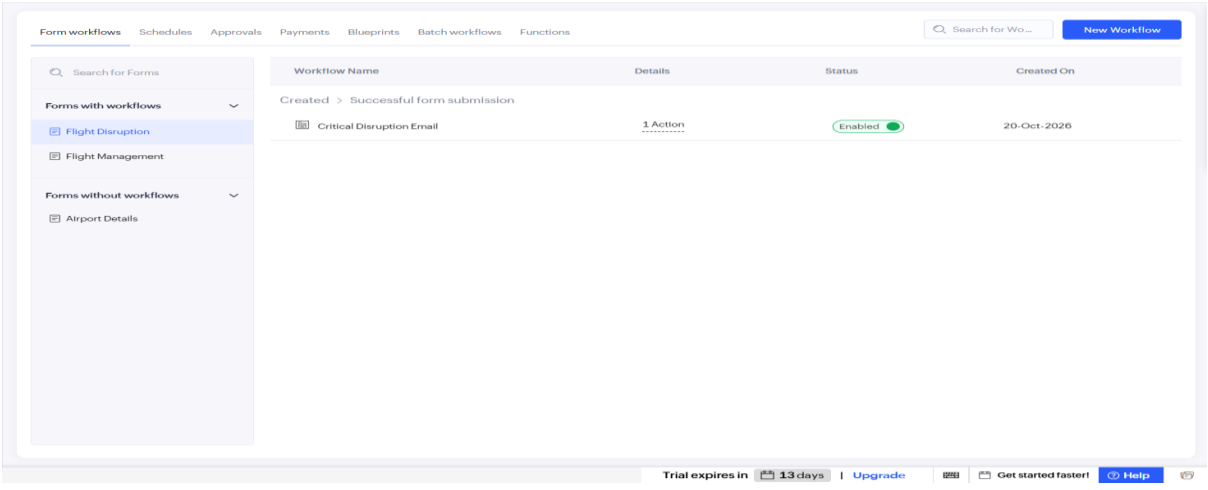
5.6 Workflows and Automation

Workflow configuration can reduce repetitive data-entry tasks and help maintain consistency between related fields. In the Airport Chaos Monitoring System, the available project information identifies workflow logic intended to copy disruption severity into the Priority field.
The high-priority unresolved disruptions report uses its configured criteria to identify records requiring attention. The report and workflow serve different purposes: the workflow supports field-level automation, while the report presents records that meet the specified conditions. The exact trigger conditions and end-to-end workflow execution should be tested before claiming that all automation scenarios work correctly.

Table 5.3: Workflows and Automation

Workflow Name	Type	Trigger / Event	Condition	Action	Status
Disruption Severity to Priority	Field-related workflow logic	Exact trigger to be verified	Disruption severity is available	Intended to copy severity into the Priority field	Configuration reported; execution requires verification
High-Priority Open Disruptions	Report filtering	Report retrieval	Record meets the configured priority and unresolved-status criteria	Displays matching disruption records	DSP002 observed in the demonstration report
Additional Notifications	Not verified	Not verified	Not verified	No notification action claimed	Not verified

Figure 5.7: Disruption Workflow or Report Configuration



5.7 Integrations

The current prototype uses Zoho Creator's built-in forms, reports, dashboard components, and configured application logic. No external flight-data API or other external service integration has been verified.

The application therefore demonstrates basic airport operations monitoring using stored demonstration records rather than automatic retrieval of live flight information.

Table 5.4: Integrations Implemented

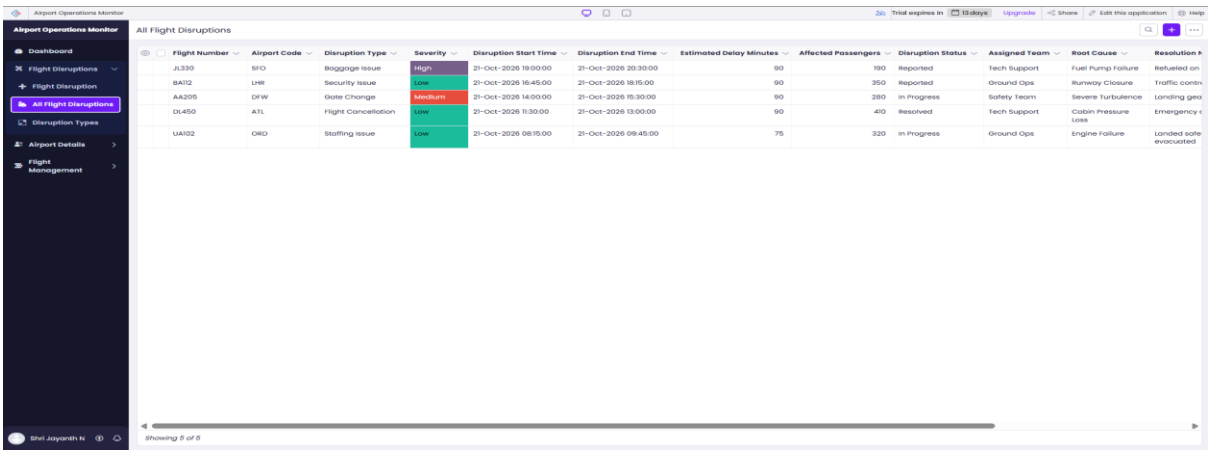
Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Zoho Creator forms and reports	Built-in application components	Form submission and report retrieval	Zoho Creator account access	Store and display airport, flight, and disruption records
External flight-data API	None implemented	Not applicable	Not applicable	Live flight information is not retrieved
External webhook or REST API	None verified	Not applicable	Not applicable	No external API exchange claimed
External notification service	None verified	Not applicable	Not applicable	No external notification integration claimed

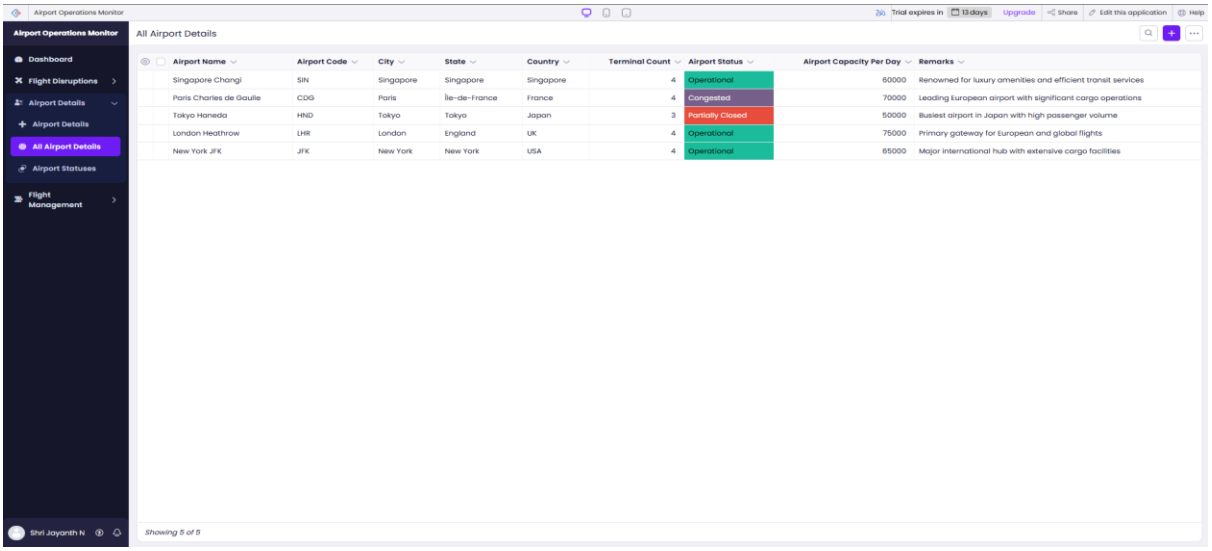
5.8 User Management and Role-Based Access

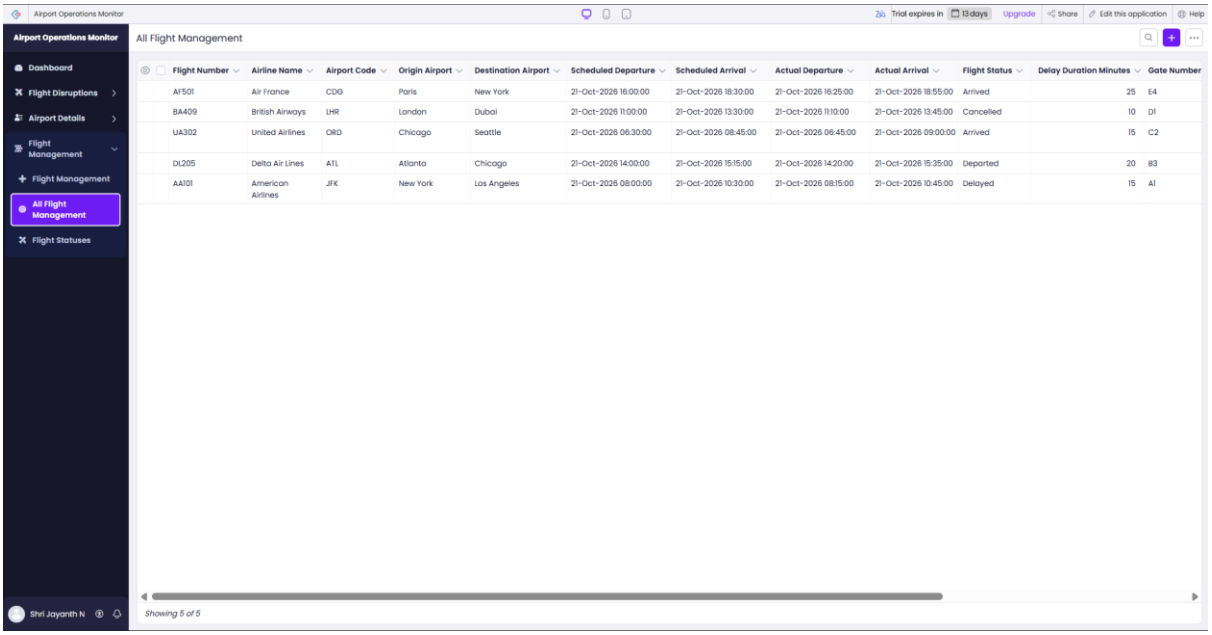
The application is intended to support airport operations personnel with different responsibilities, such as maintaining airport information, reviewing flight statuses, and monitoring disruptions.

However, the existence of conceptual roles does not establish that separate user permission profiles have been configured. Actual Zoho Creator permissions for forms, reports, dashboard pages, and individual records must be verified before the application is used with operational data.

Figure 5.8: Application Access or User Management







5.9 AI and Analytics Features

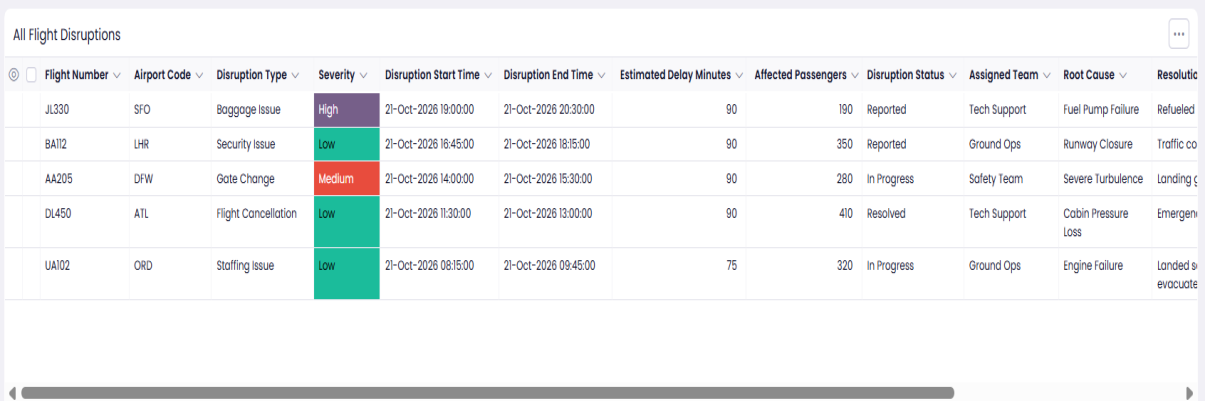
The current prototype provides basic operational summaries through dashboard statistic cards and a filtered disruption report. These features support record monitoring and identification of high-priority unresolved issues.

The dashboard summarizes total airports, total flights, delayed flights, and total disruptions.

The filtered report helps identify disruptions that meet the configured priority and resolution-status criteria.

No verified AI-based prediction, machine-learning model, or live flight analytics feature is claimed in this implementation. Advanced analytics, automatic delay prediction, and trend analysis are considered future enhancements.

Figure 5.9: Dashboard Metrics and Disruption Monitoring



All Flight Disruptions

Flight Number	Airport Code	Disruption Type	Severity	Disruption Start Time	Disruption End Time	Estimated Delay Minutes	Affected Passengers	Disruption Status	Assigned Team	Root Cause	Resolutic
JL330	SFO	Baggage Issue	High	21-Oct-2026 19:00:00	21-Oct-2026 20:30:00	90	190	Reported	Tech Support	Fuel Pump Failure	Refueled
BA112	LHR	Security Issue	Low	21-Oct-2026 16:45:00	21-Oct-2026 18:15:00	90	350	Reported	Ground Ops	Runway Closure	Traffic co
AA205	DFW	Gate Change	Medium	21-Oct-2026 14:00:00	21-Oct-2026 15:30:00	90	280	In Progress	Safety Team	Severe Turbulence	Landing g
DL450	ATL	Flight Cancellation	Low	21-Oct-2026 11:30:00	21-Oct-2026 13:00:00	90	410	Resolved	Tech Support	Cabin Pressure Loss	Emergen
UA102	ORD	Staffing Issue	Low	21-Oct-2026 08:15:00	21-Oct-2026 09:45:00	75	320	In Progress	Ground Ops	Engine Failure	Landed s evacuate

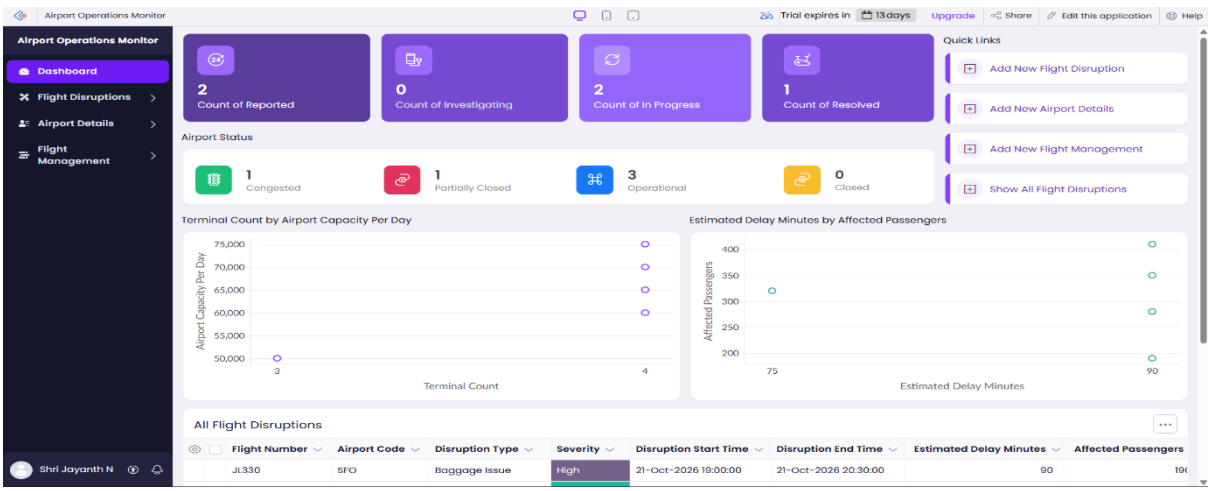
5.10 Pro-Code Extension

The Airport Chaos Monitoring System is implemented as a low-code prototype using Zoho Creator's visual development components and configured application logic. No separate pro-code extension, external custom service, or Creator REST API integration has been verified. For the current project scope, forms, reports, dashboard components, and workflow configuration provide the basic functionality required for demonstration. If future requirements include live flight-data integration, advanced predictive analytics, or complex external processing, an additional coded service or API integration could be evaluated. Such extensions are proposed future work and are not part of the verified implementation.

5.11 Mobile and Portal Access

The available project information documents the application as a Zoho Creator web-based prototype. The forms, reports, and dashboard are intended to support access to airport, flight, and disruption records through the application interface.
Mobile-specific layout behavior, offline functionality, external customer portals, and mobile-device compatibility have not been independently tested or verified. These capabilities should be evaluated separately if they become requirements for a future operational version.

Figure 5.10: Airport Chaos Monitoring System Web Interface



CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing is an important stage in the development of the Airport Chaos Monitoring System. It helps verify whether the application forms, reports, dashboard metrics, and configured workflow logic behave as expected.
The testing strategy covers airport record entry, flight information entry, disruption record creation, dashboard metric calculation, report filtering, workflow behavior, and application access permissions.
The available demonstration confirms that sample airport, flight, and disruption records are displayed in the application. The dashboard displays summary counts, and the High-Priority Open Disruptions report displays the DSP002 record. However, these observations do not constitute a complete test execution log.
Therefore, the test cases in this chapter include expected results and recorded observations where available. Tests that have not been executed or independently verified are marked accordingly.

The following testing methods are considered:

- **Form Testing:** Verify that airport, flight, and disruption records can be entered and saved correctly.
- **Report Testing:** Verify that reports display the correct records and apply their configured criteria.
- **Dashboard Testing:** Verify that summary metrics correspond to the underlying application records.
- **Workflow Testing:** Verify that disruption severity is copied to the Priority field as intended.
- **Validation Testing:** Check how forms respond to missing or invalid input.
- **Access Testing:** Verify that users can access only the forms, reports, and records permitted by their configured permissions.

6.2 Test Cases

The test cases are derived from the functional requirements described in Chapter 3. The actual results are based only on available demonstration observations. A test is not marked as passed merely because the corresponding form or feature exists.

Table 6.1: Functional and Workflow Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Verify airport record management	Enter airport name, code, location, operational status, and capacity; submit the record.	Airport record is saved and appears in the Airport Details report.	A demonstration airport record is represented in the application; a separate execution log is unavailable.	Partially evidenced

TC-02	FR-02	Verify flight record management	Enter flight number, airline, origin, destination, scheduled departure, actual departure, and status.	Flight record is saved and appears in the Flight Management report.	A demonstration flight record, including AI573, is available in the project information.	Partially evidenced
TC-03	FR-03	Verify disruption record creation	Enter disruption ID, flight number, disruption type, severity, date, and description.	Disruption record is saved and appears in the disruption report.	Sample disruption records DSP001 and DSP002 are documented.	Evidence available
TC-04	FR-05	Verify dashboard metrics	Open the Airport Chaos Dashboard and review all four statistic cards.	Cards display counts corresponding to the underlying airport, flight, delayed-flight, and disruption records.	All four cards displayed a count of 1 during the recorded demonstration.	Observed
TC-05	FR-06	Verify disruption report filtering	Open the High-Priority Open Disruptions report.	Only records meeting the configured criteria are displayed.	DSP002 appeared in the report during the demonstration.	Observed
TC-06	FR-04	Verify disruption status updates	Update a disruption's resolution status and review the record.	The updated status is saved and reflected in the relevant report.	No separate status-update test result was recorded.	Not tested
TC-07	FR-07	Verify priority workflow logic	Update a disruption's severity and inspect the Priority field.	The Priority field receives the intended severity value.	Workflow logic is reported as configured, but successful execution was not independently verified.	Not verified
TC-08	NFR-02	Verify access	Sign in using	Access follows the configured	Separate role permissions and	Not tested

		permissions	different authorized user accounts and attempt permitted and restricted actions.	permissions.	access test results were not verified.	

The table shows the distinction between observed demonstration output and tests requiring further execution. A complete validation process should include the test date, tester, test data, actual result, and supporting screenshot or log for each executed case.

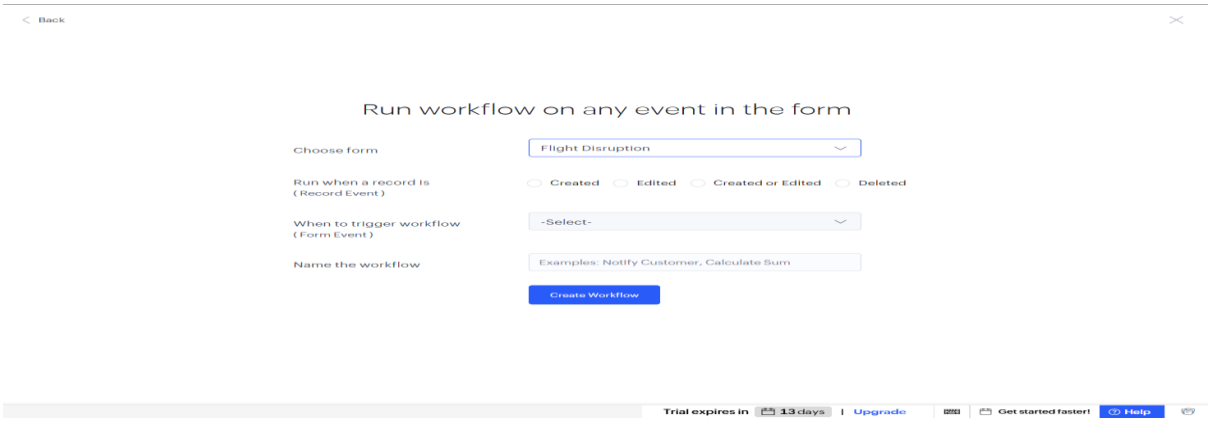
6.3 Debugging Deluge Scripts

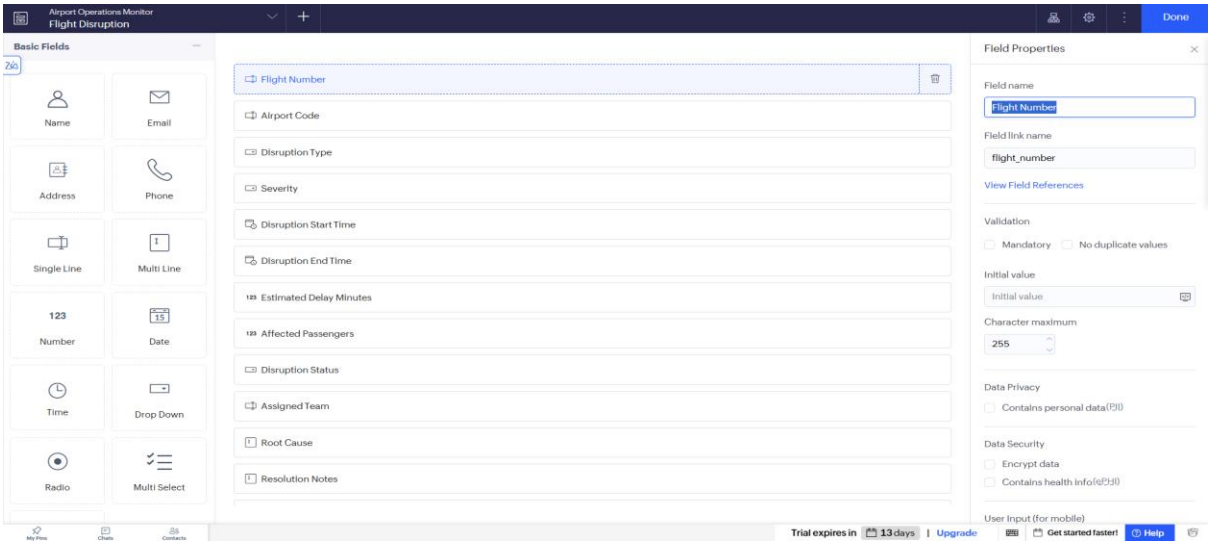
Zoho Creator supports Deluge scripting for application logic and workflow automation. In the Airport Chaos Monitoring System, the available project information identifies workflow logic intended to copy disruption severity into the Priority field.
Debugging should focus on confirming the workflow trigger, checking the source and destination field names, verifying that the correct record is updated, and examining the execution result.
The following debugging steps are recommended:

1. Open the relevant workflow configuration in Zoho Creator.
2. Verify the trigger event and the conditions under which the workflow should run.
3. Confirm that the severity field contains the expected value.
4. Check that the Priority field receives the corresponding value after the workflow executes.
5. Review any available execution logs or error messages.
6. Test the workflow with different severity values and confirm that unrelated records are not modified.
7. Record each identified defect, the cause, the correction, and the result of retesting.

No specific Deluge error, debugging output, or confirmed code correction is claimed because a corresponding debugging log was not available.

Figure 6.1: Workflow Configuration or Form Validation in Zoho Creator





6.4 Defect Log

A defect log records problems identified during testing, their causes, the corrections applied, and the results of retesting. The available project information does not provide a completed defect log. Therefore, no specific defect or fix is reported as completed.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
Not recorded	No confirmed defect report is available.	Not determined	No confirmed correction documented.	No defect record supplied
To be assigned	Verify that the Priority field updates correctly after a severity change.	To be determined through testing.	Apply a correction only if a defect is identified.	Testing required
To be assigned	Verify that resolved disruptions no longer appear in the high-priority unresolved report when they no longer meet its criteria.	To be determined through testing.	Review and correct the report criteria if necessary.	Testing required

The additional entries are proposed verification tasks, not confirmed defects. They should be updated after testing is completed.

6.5 User Acceptance Testing

User Acceptance Testing (UAT) evaluates whether the application supports the intended users and their operational tasks. For the Airport Chaos Monitoring System, UAT should focus on maintaining airport information, reviewing flight details, recording disruptions, monitoring dashboard metrics, and identifying high-priority unresolved issues.
Representative users should complete these tasks using the application and provide feedback on usability, data accuracy, navigation, and report usefulness.
No completed UAT feedback or signed acceptance record was supplied for this project. The

following table can be completed after users perform the relevant tasks.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Not recorded	Airport Operations Staff	UAT feedback not supplied.	Conduct and document UAT.
Not recorded	Operations Supervisor	UAT feedback not supplied.	Conduct and document UAT.
Not recorded	Monitoring Staff	UAT feedback not supplied.	Conduct and document UAT.

The application should undergo functional testing, permission verification, workflow validation, and user acceptance testing before being considered for operational use. The current results demonstrate selected prototype outputs but do not establish that every requirement has passed testing.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

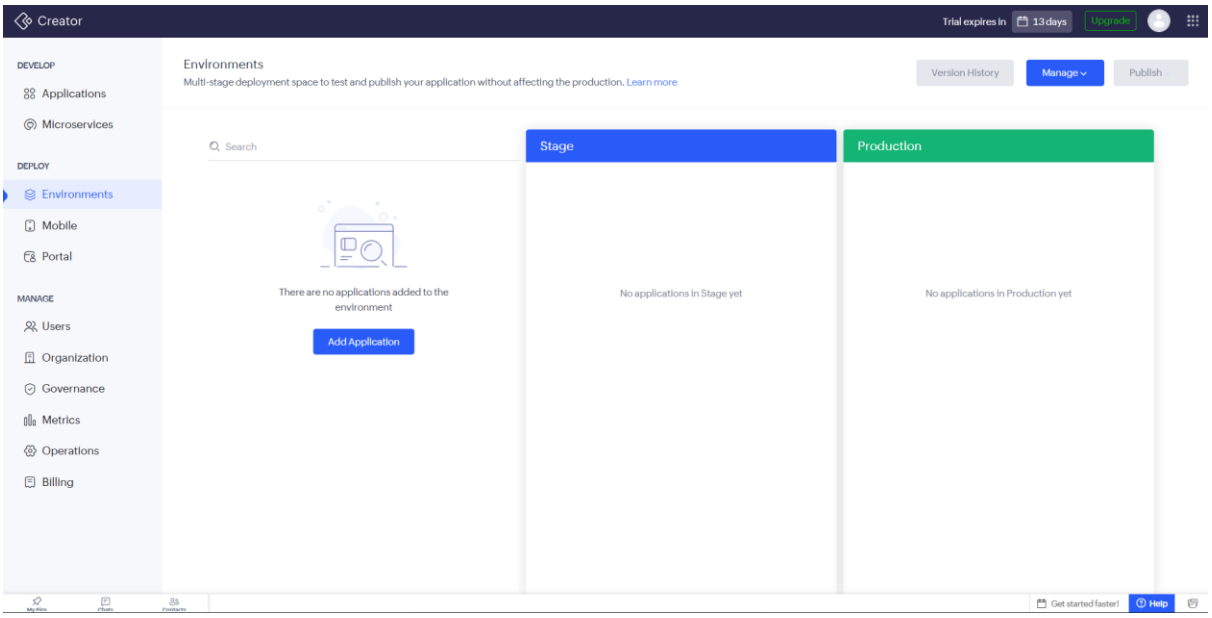
7.1 Environments and Deployment Strategy

The Airport Chaos Monitoring System is developed as a prototype using Zoho Creator. The application contains forms for airport details, flight management, and flight disruptions, together with reports and a dashboard for monitoring the stored information.
Before deployment, the application should be reviewed to confirm that the forms, report filters, dashboard metrics, and workflow logic operate as expected. The configuration should also be checked to ensure that user permissions and data-protection requirements are appropriate for the intended use.
The current project uses demonstration data and is not connected to verified live flight information. Therefore, the prototype should not be considered a production-ready airport monitoring system.
A suitable deployment strategy includes the following steps:

1. Verify the application forms, reports, dashboard, and workflow configuration.
2. Test record creation, editing, report filtering, and dashboard calculations.
3. Review user permissions and data-protection settings.
4. Confirm the availability of the required Zoho Creator account and application access.
5. Perform user acceptance testing before operational deployment.
6. Maintain a documented process for future changes and updates.

The availability of separate development, testing, and production environments, as well as any publishing or staging configuration, has not been verified.

Figure 7.1: Zoho Creator Application Environment or Deployment Configuration



7.2 Performance Optimisation

Performance optimisation helps ensure that forms, reports, and dashboard components remain responsive as the amount of stored information increases.
The Airport Chaos Monitoring System displays summary metrics and a filtered report of high-priority unresolved disruptions. However, no benchmark measurements or before-and-after performance tests have been recorded. Therefore, no specific improvement in response time or processing speed is claimed.
The following measures are recommended for future performance evaluation:

- Test reports using a larger and representative set of airport, flight, and disruption records.
- Review report filters to ensure that they retrieve the intended records efficiently.
- Avoid unnecessary fields and calculations in frequently used views.
- Measure dashboard loading time and report response time under normal usage.
- Repeat performance tests after significant application changes.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
No measured optimisation recorded	Airport Chaos Dashboard	Not measured	Not measured	No performance gain claimed
No measured optimisation recorded	Airport and flight reports	Not measured	Not measured	No performance gain claimed
No benchmark recorded	Application data volume	Demonstration records available	Production volume not evaluated	Further measurement required

The application should be evaluated with representative data volumes before it is used for operational monitoring.

7.3 Security Features

Security is important because the application stores airport information, flight details, and disruption records. Appropriate security controls should be reviewed before the application is used with operational data.
Zoho Creator provides application configuration options for user access and data handling. However, the availability of a configuration option does not establish that the corresponding security control has been enabled or tested in this project.

Table 7.3: Security Controls and Verification Status

Security Control	Where Applied	Purpose / Threat Mitigated	Status
User permissions	Forms, reports, and dashboard	Restrict access to authorized users	Configuration not verified
Data privacy settings	Fields containing sensitive information	Reduce unnecessary exposure of sensitive data	Project-specific configuration not verified

Secure account access	Zoho Creator application	Prevent unauthorized application access	Account-level configuration not audited
Workflow identity	Configured application workflow, if applicable	Avoid unnecessary hard-coded sender credentials and support traceability	Specific configuration not verified
Audit and record history	Application records	Support accountability and review of changes	Configuration not verified
Data backup and recovery	Application data	Support recovery from accidental loss or other incidents	Backup arrangements not verified

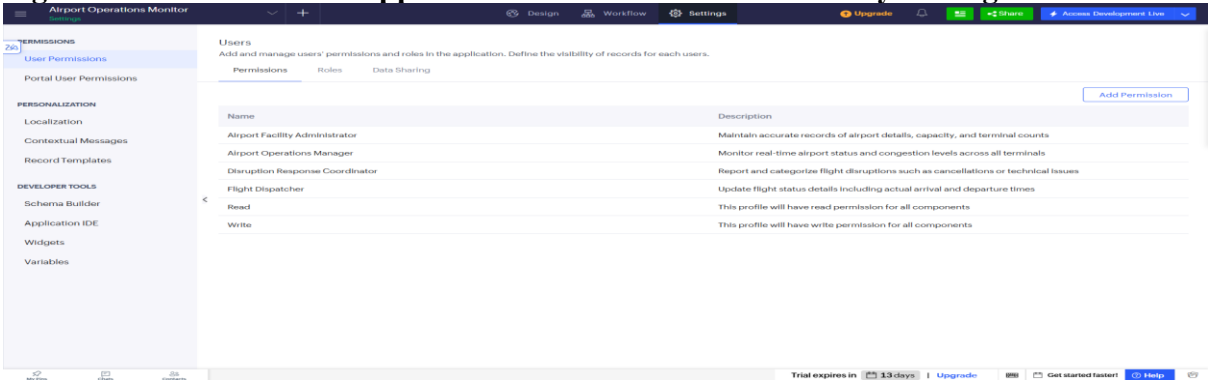
The application should follow the principle of least privilege, meaning users should receive only the permissions necessary for their responsibilities. Any sensitive operational information should be protected through suitable access controls and the platform's available security features.
The security configuration should be reviewed before deployment, and any applicable organizational requirements should be documented.

7.4 Governance and Administration

Governance defines how the application is maintained, monitored, and managed throughout its lifecycle. For the Airport Chaos Monitoring System, governance includes maintaining consistent form and field names, reviewing report criteria, verifying workflow behavior, managing user permissions, and documenting application changes.
The application currently serves as a demonstration prototype. The available project information does not confirm production ownership, backup schedules, formal change approvals, audit-trail settings, or a completed license review.
The following administrative practices are recommended:

1. Assign an application owner responsible for configuration and maintenance.
2. Document changes to forms, reports, dashboard components, and workflows.
3. Review user access whenever responsibilities change.
4. Maintain an appropriate backup and recovery procedure.
5. Test important changes before making them available to users.
6. Review the Zoho Creator account and licensing requirements before deployment.
7. Record incidents, defects, corrective actions, and approval decisions.
8. Verify the accuracy and source of operational data before using it for airport decisions.

Figure 7.2: Zoho Creator Application Administration or Security Settings



CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

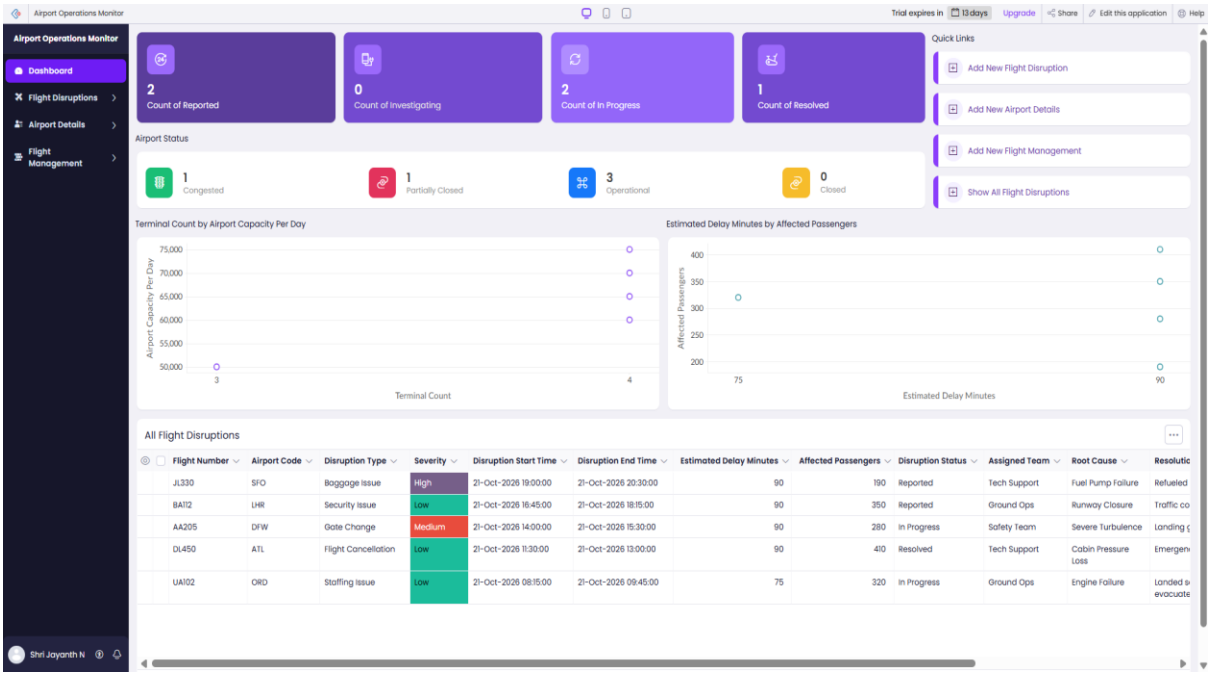
The Airport Chaos Monitoring System was developed as a prototype using Zoho Creator to organize airport details, manage flight information, record flight disruptions, and monitor operational issues through reports and a dashboard.

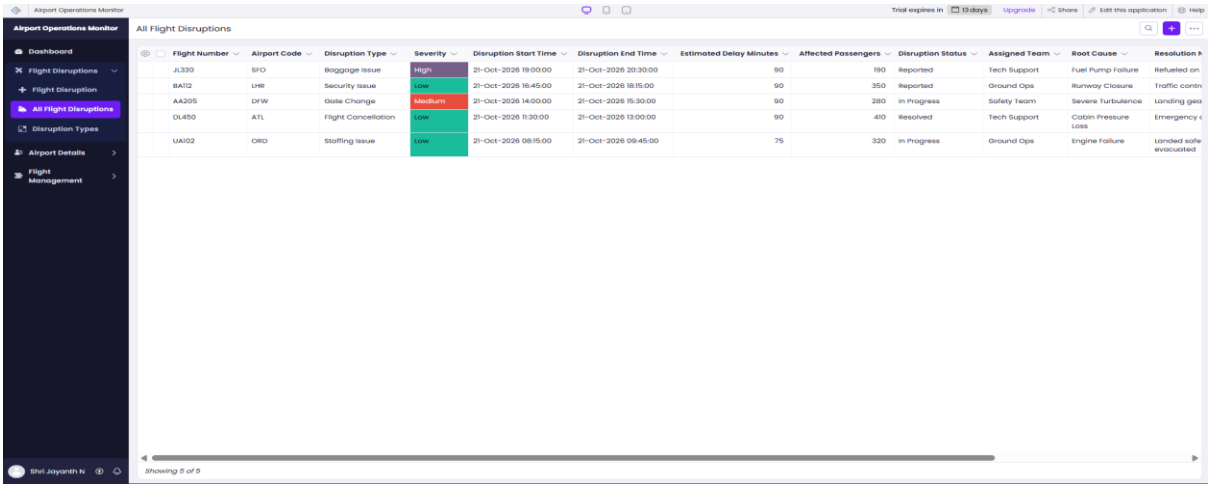
The application contains three main forms: Airport Details, Flight Management, and Flight Disruption. The Airport Details form stores basic airport information, while the Flight Management form records flight details, scheduled and actual departure times, delay duration, flight status, and disruption causes. The Flight Disruption form records disruption type, severity, description, resolution status, and action taken.

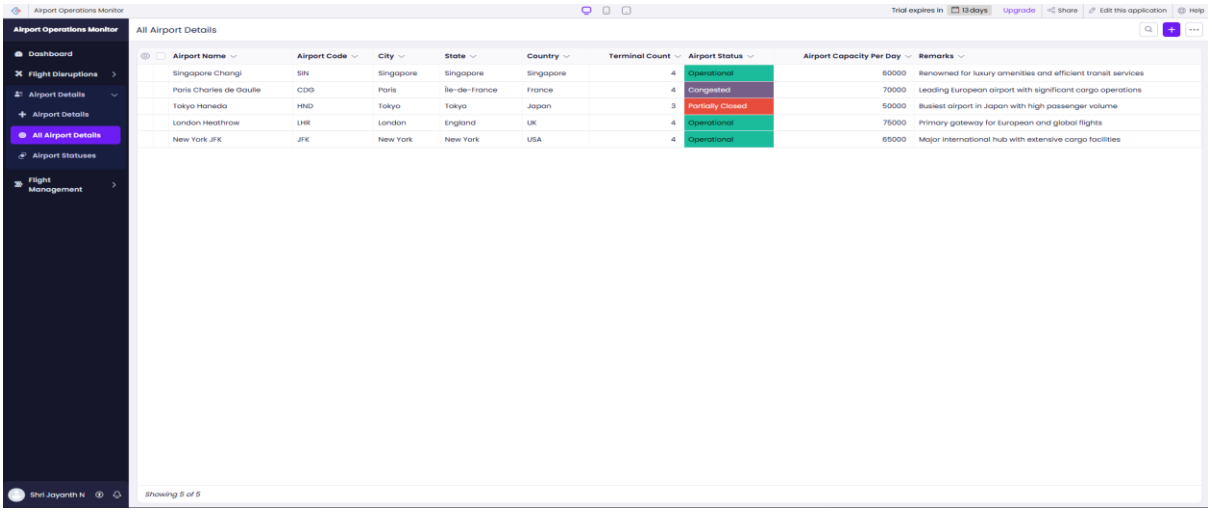
The Airport Chaos Dashboard provides an overview of the records available in the application. It contains four summary cards: Total Airports, Total Flights, Delayed Flights, and Total Disruptions. During the demonstration, each card displayed a value of 1 based on the sample data entered into the application.

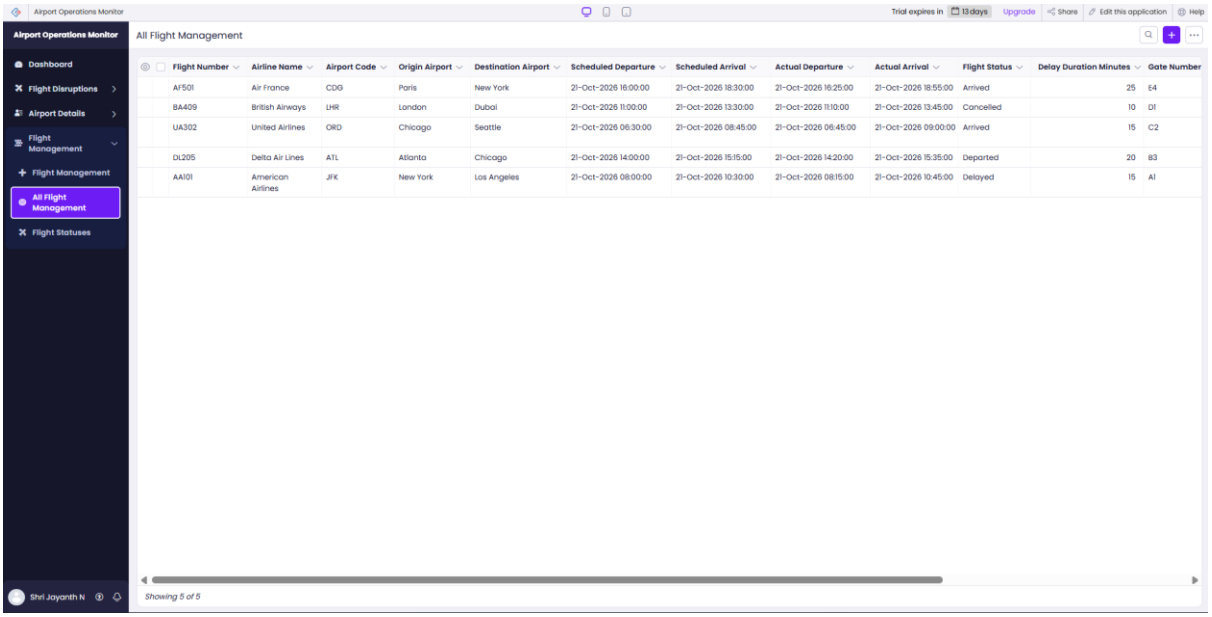
The application also includes a report named High_Priority_Open_Disruptions, which displays high-priority disruptions that remain unresolved according to its configured criteria. The sample disruption record DSP002 was displayed in this report during the demonstration.

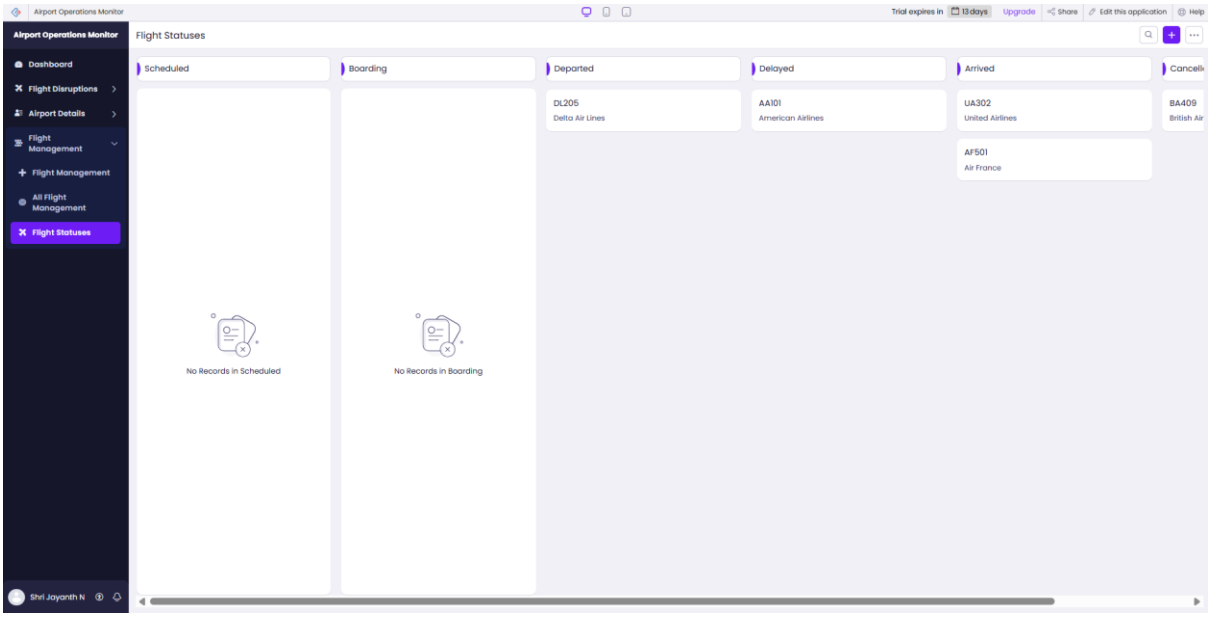
The results demonstrate that Zoho Creator can be used to organize sample airport operational data and present important disruption information through a centralized interface. However, the prototype uses demonstration data and is not connected to a verified live airport or flight data source.




Airport Operations Monitor
All Flight Disruptions
Dashboard
Flight Disruptions
Flight Disruption
All Flight Disruptions
Disruption Types
Airport Details
Flight Management
Flight Number
Airport Code
Disruption Type
Severity
Disruption Start Time
Disruption End Time
Estimated Delay Minutes
Affected Passengers
Disruption Status
Assigned Team
Root Cause
JL330 SFO Baggage Issue High 21-Oct-2026 19:00:00 21-Oct-2026 20:30:00 90 190 Reported Tech Support Fuel Pump Failure
BAI12 LHR Security Issue Low 21-Oct-2026 16:45:00 21-Oct-2026 18:15:00 90 350 Reported Ground Ops Runway Closure
AA205 DFW Gate Change Medium 21-Oct-2026 14:00:00 21-Oct-2026 15:30:00 90 280 In Progress Safety Team Severe Turbulence
DL450 ATL Flight Cancellation Low 21-Oct-2026 11:30:00 21-Oct-2026 13:00:00 90 410 Resolved Tech Support Cabin Pressure Loss
UAI02 ORD Staffing Issue Low 21-Oct-2026 08:15:00 21-Oct-2026 09:45:00 75 320 In Progress Ground Ops Engine Failure
Shri Jayanth N
Showing 5 of 5


Airport Operations Monitor
All Airport Details
Dashboard
Flight Disruptions
Airport Details
Airport Details
All Airport Details
Airport Statuses
Flight Management
Airport Name
Airport Code
City
State
Country
Terminal Count
Airport Status
Airport Capacity Per Day
Remarks
Singapore Changi SIN Singapore Singapore Singapore 4 Operational 60000 Renowned for luxury amenities and efficient transit services
Paris Charles de Gaulle CDG Paris Île-de-France France 4 Congested 70000 Leading European airport with significant cargo operations
Tokyo Haneda HND Tokyo Tokyo Japan 3 Partially Closed 50000 Busiest airport in Japan with high passenger volume
London Heathrow LHR London England UK 4 Operational 75000 Primary gateway for European and global flights
New York JFK JFK New York New York USA 4 Operational 65000 Major international hub with extensive cargo facilities
Shri Jayanth N
Showing 5 of 5


Airport Operations Monitor
All Flight Management
Dashboard
Flight Disruptions
Airport Details
Flight Management
Flight Management
All Flight Management
Flight Statuses
Flight Number
Airline Name
Airport Code
Origin Airport
Destination Airport
Scheduled Departure
Scheduled Arrival
Actual Departure
Actual Arrival
Flight Status
Delay Duration Minutes
Gate Number
AF501 Air France CDG Paris New York 21-Oct-2026 16:00:00 21-Oct-2026 18:30:00 21-Oct-2026 16:25:00 21-Oct-2026 18:55:00 Arrived 25 E4
BA409 British Airways LHR London Dubai 21-Oct-2026 11:00:00 21-Oct-2026 13:30:00 21-Oct-2026 11:10:00 21-Oct-2026 13:45:00 Cancelled 10 D1
UA302 United Airlines ORD Chicago Seattle 21-Oct-2026 06:30:00 21-Oct-2026 08:45:00 21-Oct-2026 06:45:00 21-Oct-2026 09:00:00 Arrived 15 C2
DL205 Delta Air Lines ATL Atlanta Chicago 21-Oct-2026 14:00:00 21-Oct-2026 15:15:00 21-Oct-2026 14:20:00 21-Oct-2026 15:35:00 Departed 20 B3
AA101 American Airlines JFK New York Los Angeles 21-Oct-2026 08:00:00 21-Oct-2026 10:30:00 21-Oct-2026 08:15:00 21-Oct-2026 10:45:00 Delayed 15 A1
Shri Jayanth N
Showing 5 of 5



8.2 Achievement of Objectives

The project objectives were evaluated based on the forms, reports, dashboard, and sample records available in the Zoho Creator prototype. Table 8.1 presents the relationship between the objectives and the demonstrated outcomes.

Table 8.1: Mapping of Objectives to Outcomes

Objective	Implemented Through	Outcome / Evidence
Maintain structured airport records	Airport Details form	Airport information can be recorded in a centralized form.
Manage flight information	Flight Management form	Flight details, status, delay duration, and disruption cause are represented in the form.
Record and monitor flight disruptions	Flight Disruption form	Disruption type, severity, resolution status, and action taken can be recorded.
Display operational summaries	Airport Chaos Dashboard	Four summary cards display airport, flight, delayed-flight, and disruption counts. Each displayed 1 during the demonstration.
Identify high-priority unresolved disruptions	High_Priority_Open_Disruptions report	Sample record DSP002 appeared in the report during the demonstration.
Explore workflow-based automation	Severity-to-priority workflow configuration	The intended workflow logic was considered in the implementation; successful execution requires verification.

The project achieved its main prototype objectives by providing structured forms, a dashboard, and a report for monitoring sample flight disruptions. Live-data integration, comprehensive automated testing, and successful execution of all workflow configurations have not been established as completed outcomes.

8.3 Comparison with the Existing System

In a conventional approach, airport operational information may be maintained through separate spreadsheets, documents, or manually prepared reports. The proposed prototype demonstrates an alternative in which related information is recorded and viewed through Zoho Creator forms and reports. The existing-system descriptions in Table 8.2 represent a general comparison, not a measured assessment of a specific airport's current process.

Table 8.2: Existing System versus Proposed Application

Parameter	Conventional Approach	Proposed Application	Observed Improvement
Record management	Information may be maintained in separate files or records.	Airport, flight, and disruption information is represented through dedicated forms.	Structured record entry is demonstrated; efficiency gains were not measured.
Flight-delay monitoring	Delays may require manual review of available records.	Flight status and delay duration are recorded in the Flight Management form.	Delay information is organized in one application; live monitoring is not available.
Disruption tracking	Disruption details may be recorded separately.	The Flight Disruption form stores severity, status, and action taken.	A consistent record structure is provided.
Report generation	Summaries may need to be prepared manually.	Creator reports include a high-priority open-disruptions view.	A predefined report is demonstrated; time saved was not measured.
Operational overview	Information may need to be reviewed across multiple records.	The dashboard displays four summary metrics.	A centralized overview is demonstrated using sample data.
Data accessibility	Access depends on the tools and sharing arrangements in use.	Records are accessed through the Zoho Creator application.	A common interface is provided; role-based permissions require verification.

The comparison indicates that the proposed application provides a structured method for recording and reviewing airport operational information. However, no quantitative comparison of processing time, error rates, or operational efficiency was conducted. Therefore, the results demonstrate the application's functionality rather than proving a measured improvement over an existing airport system.

8.4 Limitations of the Low-Code Approach

The project demonstrates the benefits of developing a prototype with Zoho Creator, but several limitations should be considered.

1. **Platform dependency:** The application relies on Zoho Creator forms, reports, workflows, pages, and its Deluge scripting environment. Moving the application to another platform may require redesign or redevelopment.
2. **Live-data integration:** The current prototype uses sample data and does not connect to a verified live flight information source. Therefore, it cannot automatically obtain real-time flight schedules or delays.
3. **Workflow verification:** The intended severity-to-priority workflow requires testing to confirm that it executes correctly under the configured conditions.
4. **Advanced analytics:** The current dashboard displays summary counts and a filtered disruption report. Predictive analysis, trend forecasting, and advanced operational analytics have not been implemented.
5. **Security and access control:** User roles and permissions must be configured and verified before the application can be used for operational or sensitive information.
6. **Scalability and performance:** The prototype has not been evaluated using large datasets or under heavy concurrent usage. Its performance in a production airport environment is therefore unknown.
7. **Availability and licensing:** Access to the application depends on the relevant Zoho Creator account, service availability, and licensing conditions. A trial-expiration message was observed during development, but its effect on continued access and functionality was not fully verified.

These limitations do not prevent the prototype from demonstrating the basic concept of centralized airport disruption monitoring. However, additional development, testing, integration, and security verification would be required before considering production deployment.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The Airport Chaos Monitoring System illustrates how different development approaches can be combined within a low-code platform.

No-code development: The visual creation of forms, reports, dashboard components, and field options reduces the need to write code for basic data entry and presentation.

Low-code development: Zoho Creator supports workflow configuration and Deluge scripting for business logic. In this project, severity-to-priority automation was considered as part of the implementation, but its successful execution must be verified before it can be treated as a confirmed result.

Pro-code development: Future requirements such as custom API services, advanced integrations, specialized user interfaces, or complex analytics may require conventional programming or external services. These are possible extensions and are not features currently demonstrated by the prototype.

The project shows that low-code development is suitable for building an initial application with structured records, reports, and dashboard summaries. The choice between no-code, low-code, and pro-code development should depend on the required functionality, integration needs, security requirements, scalability, and long-term maintenance.

Overall, the prototype provides a foundation for organizing airport information and monitoring sample flight disruptions. Future enhancements can build on this foundation to support more comprehensive operational monitoring.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Airport Chaos Monitoring System was developed as a prototype using Zoho Creator to organize airport information, manage flight details, record flight disruptions, and monitor operational issues through reports and a dashboard.

The application includes three main forms: Airport Details, Flight Management, and Flight Disruption. These forms provide a structured way to record airport information, flight status, delay duration, disruption severity, resolution status, and action taken. The Airport Chaos Dashboard presents summary counts for total airports, total flights, delayed flights, and total disruptions. The High_Priority_Open_Disruptions report helps identify high-priority disruptions that remain unresolved according to its configured criteria.

The demonstration showed that the application can organize sample airport data and display important information through a centralized interface. It also provided practical experience in designing forms, configuring reports, creating dashboard components, and exploring workflow-based automation in Zoho Creator.

However, the prototype uses demonstration data and is not connected to a verified live flight information source. Further testing of workflow execution, access permissions, security, and performance is necessary before the application can be considered for operational use.

In conclusion, the project demonstrates the usefulness of low-code application development for creating an initial airport operations monitoring solution. It provides a foundation for future enhancements involving live flight information, automated alerts, advanced analytics, and improved operational monitoring.

9.2 Future Enhancements

The following enhancements can improve the functionality, reliability, and practical usefulness of the Airport Chaos Monitoring System.

1. **Live Flight Data Integration:** Integrate a reliable flight information API to obtain updated flight schedules, departure times, arrival times, and delay information, subject to API availability and access permissions.
2. **Automated Notifications:** Configure and verify notifications for high-severity disruptions, significant flight delays, and unresolved operational issues so that responsible staff can respond promptly.
3. **Advanced Dashboard and Analytics:** Introduce charts and trend reports to analyze flight delays, disruption categories, severity levels, and resolution status over selected periods.
4. **Workflow Automation:** Complete testing of the severity-to-priority workflow and introduce additional automation for assigning disruptions, updating statuses, and tracking follow-up actions.
5. **Role-Based Access Control:** Configure and verify suitable permissions for administrators, airport operations staff, supervisors, and monitoring staff to ensure that users can access only the information and functions required for their responsibilities.
6. **Resolution Time Tracking:** Record the time at which a disruption is reported and resolved. This would help analyze resolution duration and identify recurring operational problems.
7. **Mobile Accessibility:** Evaluate mobile access so authorized personnel can review

flight information and disruption records while working in different airport areas.

8. **Security, Backup, and Recovery:** Establish and test appropriate access controls, backup procedures, data protection practices, and recovery arrangements before any production deployment.
9. **Performance and Scalability Testing:** Evaluate the application using larger datasets and multiple users to understand its performance and identify any required improvements.

These enhancements are proposed future developments. They have not been treated as completed features of the current prototype.

9.3 Learning Outcomes and Career Relevance

The development of the Airport Chaos Monitoring System provided practical exposure to low-code application development using Zoho Creator. The project helped demonstrate how a business problem can be translated into structured forms, reports, dashboard components, and workflow requirements.

The key learning outcomes include:

- **Form Design and Data Management:** Understanding how to organize airport, flight, and disruption information into separate forms with clearly defined fields.
- **Report Development:** Learning how reports can present stored information and filter high-priority unresolved disruptions.
- **Dashboard Design:** Understanding how summary cards can provide a quick overview of important operational indicators.
- **Workflow Concepts:** Exploring how business rules and workflow automation can support disruption monitoring and priority management.
- **System Analysis and Design:** Identifying stakeholders, functional requirements, non-functional requirements, system components, and possible future improvements.
- **Testing and Validation:** Recognizing the importance of verifying form behavior, report criteria, workflow execution, data accuracy, and access permissions.
- **Security and Governance:** Understanding the need for appropriate user access, data protection, backup procedures, and controlled deployment.
- **Problem-Solving:** Applying low-code development concepts to create a prototype addressing airport operational monitoring requirements.

The skills developed through this project are relevant to careers in low-code application development, business analysis, workflow automation, software testing, and application support. Further learning in API integration, database design, security, and advanced programming would help extend the prototype into a more comprehensive operational solution. Overall, the project strengthened the understanding of how low-code platforms can be used to develop structured business applications and demonstrated the importance of testing, security, and continuous improvement in software development.

REFERENCES

References are listed in IEEE style and cited in the report.

[1] D. Gao, F. Fagerholm, and V. Toivanen, "What does current research say about the viability of low-code development? A systematic literature review," *Journal of Systems and Software*, vol. 239, Art. no. 112893, 2026, doi: 10.1016/j.jss.2026.112893.

[2] M. O. Ajimati, N. Carroll, and M. Maher, "Adoption of low-code and no-code development: A systematic literature review and future research agenda," *Journal of Systems and Software*, vol. 222, Art. no. 112300, 2025, doi: 10.1016/j.jss.2024.112300.

[3] Zoho Corporation, "A Quickstart Guide to Zoho Creator," *Zoho Creator Help*. [Online]. Available: https://www.zoho.com/creator/help/new-quickstart-guide.html. Accessed: Oct. 21, 2026.

[4] Zoho Corporation, "Understand Application," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/creating-applications/understand-applications/articles/understand-application. Accessed: Oct. 21, 2026.

[5] Zoho Corporation, "Understanding Reports," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/reports/understand-reports/articles/understand-reports. Accessed: Oct. 21, 2026.

[6] Zoho Corporation, "Understanding Pages," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/pages/create-and-manage-pages/articles/understand-pages. Accessed: Oct. 21, 2026.

[7] Zoho Corporation, "Understanding Workflows," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/understand-workflows/articles/understand-workflows. Accessed: Oct. 21, 2026.

[8] Zoho Corporation, "Creating and Managing Form Workflows," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/workflows/create-and-manage-form-workflows/articles/create-new-form-workflow. Accessed: Oct. 21, 2026.

[9] Zoho Corporation, "Understanding Page Builder," *Zoho Creator Help*. [Online]. Available: https://help.zoho.com/portal/en/kb/creator/developer-guide/pages/create-and-manage-pages/articles/understand-page-builder. Accessed: Oct. 21, 2026.

APPENDIX A
ZOHO CREATOR FEATURE COVERAGE CHECKLIST

This checklist identifies the features implemented or observed in the Airport Chaos Monitoring System prototype. Features that have not been verified are marked accordingly.
Table A.1: Feature Coverage Checklist

S. No.	Zoho Creator Feature	Status	Location in Application	Report Section
1	Application created	Implemented	Airport Chaos Monitoring System	5.1
2	Application sections and menu organised	Partially verified	Application navigation	5.1
3	Airport Details form	Implemented	Airport Details	5.2
4	Flight Management form	Implemented	Flight Management	5.2
5	Flight Disruption form	Implemented	Flight Disruption	5.2
6	Airport records	Demonstrated	Airport Details	5.2
7	Flight records	Demonstrated	Flight Management	5.2
8	Flight disruption records	Demonstrated	Flight Disruption	5.2
9	Standard reports	Implemented	Application reports	5.3
10	High-priority open disruptions report	Implemented/observed	High_Priority_Open_Disruptions	5.3
11	Total Airports dashboard card	Observed	Airport Chaos Dashboard	5.4
12	Total Flights dashboard card	Observed	Airport Chaos Dashboard	5.4
13	Delayed Flights dashboard card	Observed	Airport Chaos Dashboard	5.4
14	Total Disruptions	Observed	Airport Chaos Dashboard	5.4

	dashboard card			
15	Deluge script for severity-to-priority update	Requires verification	Workflow/script configuration	5.5
16	Automated workflow execution	Requires verification	Workflow configuration	5.6
17	External flight API integration	Not implemented	Not applicable	5.7
18	Role-based access control	Requires verification	User and permission settings	5.8
19	AI-based flight delay prediction	Not implemented	Not applicable	5.9
20	Advanced analytics and trend charts	Future enhancement	Not implemented	5.9
21	Pro-code extension	Not implemented	Not applicable	5.10
22	Mobile application testing	Not verified	Not tested	5.11
23	Portal access	Not verified	Not tested	5.11
24	Live airport data connection	Not implemented	Not applicable	7.1
25	Security and permission testing	Requires verification	Application settings	7.3
26	Backup and recovery testing	Not verified	Not tested	7.4
27	Functional testing	Partially completed	Test cases in Chapter 6	6.2
28	User acceptance testing	Not documented	Feedback not supplied	6.5

APPENDIX B

COURSE OUTCOME MAPPING

This appendix explains how the Airport Chaos Monitoring System project relates to the course outcomes for low-code application development. The outcome descriptions should be updated according to the official syllabus provided by the institution.

Table B.1: Course Outcome Mapping

S.No.	Course Outcome	Application in the Project
1	Understand low-code application development	Zoho Creator is used to develop an application using forms, reports, pages, and workflows.
2	Analyse a real-world problem	The project addresses the need to organise airport information, flight details, and disruption records.
3	Design an application structure	The application separates airport details, flight management, and flight disruption information into forms.
4	Develop an application using a low-code platform	Zoho Creator is used to create forms, reports, and a dashboard.
5	Apply workflow automation concepts	The project considers severity-to-priority updates as an automation task requiring verification.
6	Test and validate an application	Sample records and dashboard outputs are used to examine the prototype's behaviour.
7	Identify limitations and future improvements	The report identifies live flight data integration, notifications, analytics, and security verification as areas for improvement.

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The following Deluge script checks whether the disruption severity is set to Critical. If the condition is satisfied, it retrieves the disruption details, constructs an HTML email message, and sends a critical flight disruption alert to the configured management email address.

```
 // Check if severity is Critical
if(input.severity == "Critical")
{
	// Get the disruption details
	disruptionType = input.disruption_type;
	flightNumber = input.flight_number;
	airportCode = input.airport_code;
	estimatedDelay = input.estimated_delay_minutes;
	affectedPassengers = input.affected_passengers;
	// Construct email message
	subject = "CRITICAL: Flight Disruption Alert - " + flightNumber;
	message = "<p>Dear Email Management Team,</p>";
	message = message.concat("<p>A critical flight disruption has been reported:</p>");
	message = message.concat("<ul>");
	message = message.concat("<li><strong>Flight Number:</strong> " + flightNumber +
"</li>");
	message = message.concat("<li><strong>Airport Code:</strong> " + airportCode +
"</li>");
	message = message.concat("<li><strong>Disruption Type:</strong> " +
disruptionType + "</li>");
	message = message.concat("<li><strong>Estimated Delay:</strong> " +
estimatedDelay + " minutes</li>");
	message = message.concat("<li><strong>Affected Passengers:</strong> " +
affectedPassengers + "</li>");
	message = message.concat("</ul>");
	message = message.concat("<p>Please take immediate action.</p>");
	// Send email to management team
	sendmail
	[
		from :zoho.loginuserid
		to :"emailmanagement@zoho.com"
		subject :subject
		message :message
		content type :HTML
	]
}
```

APPENDIX - D
USER MANUAL

The User Manual explains how to access and use the Airport Chaos Monitoring System developed using Zoho Creator. It provides instructions for entering airport information, managing flight details, recording flight disruptions, monitoring dashboard information, and receiving critical disruption alerts.

D.1 Accessing the Application

1. Open the Airport Chaos Monitoring System in Zoho Creator.
2. Sign in using an authorised account, if required.
3. Open the application from the available applications list.
4. Use the application menu to access the Airport Details, Flight Management, and Flight Disruption forms.
5. Open the dashboard to view airport and flight monitoring information.
6. Use the available reports to review recorded information and identify high-priority open disruptions.

Note: Access depends on the availability of the Zoho Creator application and the permissions assigned to the user.

D.2 Instructions for Airport Operations Staff

Step 1: Enter Airport Details

1. Open the Airport Details form.
2. Enter the airport name and airport code.
3. Enter the city and country.
4. Select the airport category and operational status, where applicable.
5. Enter the daily flight capacity.
6. Review the information and submit the form.

Step 2: Enter Flight Details

1. Open the Flight Management form.
2. Enter the flight number and airline name.
3. Enter the origin and destination airports.
4. Select the flight date.
5. Enter the scheduled departure time and actual departure time, where available.
6. Enter the delay duration in minutes.
7. Select the appropriate flight status.
8. Enter the disruption cause, if applicable.
9. Review the details and save the record.

Step 3: Record a Flight Disruption

1. Open the Flight Disruption form.
2. Enter the disruption ID and flight number.
3. Select the disruption type.
4. Select the appropriate severity level.
5. Enter the disruption date and description.
6. Enter the estimated delay in minutes.
7. Enter the number of affected passengers.
8. Update the resolution status and action taken, where applicable.
9. Review the information and submit the record.

Ensure that the information entered is accurate because the dashboard, reports, and alert notifications depend on the recorded data.

D.3 Instructions for Monitoring Staff and Supervisors

1. Open the Airport Chaos Dashboard.
2. Review the Total Airports card to check the number of airport records.
3. Review the Total Flights card to check the number of flight records.
4. Review the Delayed Flights card to identify the number of flights marked as delayed.
5. Review the Total Disruptions card to check the number of recorded disruptions.
6. Open the High_Priority_Open_Disruptions report to review disruptions that match the report's configured criteria.
7. Check the disruption severity, description, and resolution status.
8. Follow the organisation's procedures to coordinate the required response.
9. Update the disruption record when its status or action taken changes, if the user has permission to do so.

D.4 Critical Flight Disruption Email Alert

The application includes a Deluge script designed to send an email notification when a flight disruption has a severity level of Critical.

The following steps describe the intended alert process:

1. Open the Flight Disruption form.
2. Enter the required flight and disruption details.
3. Set the severity field to Critical.
4. Submit or update the record according to the configured workflow trigger.
5. When the configured workflow executes and the script runs successfully, an email alert is sent to the configured management email address.
6. The email subject includes the flight number.
7. The email body contains the flight number, airport code, disruption type, estimated delay, and number of affected passengers.
8. The management team can review the information and take appropriate action.

Important: Email delivery depends on the workflow configuration, correct field link names, Zoho Creator permissions, and successful execution of the sendmail task. Test the notification in the application before claiming that email delivery has been verified.

D.5 Viewing Reports

1. Open the required report from the application menu.
2. Review the displayed airport, flight, or disruption records.
3. Open the High_Priority_Open_Disruptions report to view records matching its configured criteria.
4. Select a record to review its available details, if record access is permitted.
5. Verify the resolution status before taking further action.

The information displayed in a report depends on its configuration and the records available in the application.

D.6 Troubleshooting

S. No.	Issue	Recommended Action
1	A record is not displayed in a report	Check the report criteria and confirm that the record has been saved.
2	Dashboard counts	Check the source records and the criteria used by each

	appear incorrect	dashboard card.
3	A critical alert email is not received	Verify the severity value, workflow trigger, field link names, sender configuration, and email execution status.
4	A record cannot be edited	Check whether the logged-in user has the required permissions.
5	The application cannot be accessed	Check account access, application availability, and internet connectivity.

D.7 General Usage Guidelines

1. Enter accurate and complete information in all required fields.
2. Use consistent flight numbers and airport codes when recording related information.
3. Select the correct severity level for each disruption.
4. Update the resolution status when new information becomes available.
5. Review critical disruption alerts promptly.
6. Avoid entering confidential passenger information unless it is authorised and necessary.
7. Ensure that application access and permissions are managed by an authorised administrator.

D.8 Conclusion

The Airport Chaos Monitoring System provides a centralised interface for recording airport information, managing flight details, monitoring disruptions, and reviewing dashboard reports. Its critical disruption alert script is designed to notify the configured management email address when a disruption is marked as Critical. Proper data entry, workflow configuration, access control, and testing are essential for reliable operation.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SHRI JAYANTH N	Register No.	2116241501201
Project Title	Airport Chaos Monitoring System	Date	21 October 2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder front desks	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	SHRI JAYANTH N	Register No.	2116241501201
Service Type / Section	Artificial Intelligence and Machine Learning	Date of Viva	To be completed

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**